COLLÈGE LA CITÉ · SCIENCE DES DONNÉES APPLIQUÉES

Bases de données massives avancées (IFM30522)

Projet d'équipe (UA2) · Rapport détaillé

Tableau de bord des ventes multi-magasins

De HDFS au tableau de bord : 24 000 ventes de 2025 agrégées par MapReduce Python (Hadoop Streaming) sur un cluster Hadoop de 3 nœuds, restituées dans une application Streamlit filtrable et distribuées sous la forme d'une image Docker publiée.

Remis par

Yacine Lezoul · 2748526

Adel Benhamma · 2741167

Barnabe Landry Evouna · 2751210

Mouncif Bouchachi · 2743259

Rimy Meziani · 2741840

Douyeri Lamoussa Sanogo · 2731042

Travail	En équipe (6 membres)
Professeur	Oussama Naciri
Cours	Bases de données massives avancées (IFM30522-0030-A2026)
Date de remise	9 novembre 2026
Dépôt GitHub	https://github.com/lezoul/hadoop-mapreduce-ventes-multimagasins
Image Docker	docker.io/yacinelezoul/dashboard-ventes:1.0
Docker Hub	https://hub.docker.com/r/yacinelezoul/dashboard-ventes

Sommaire

01 Présentation et correspondance avec la grille
02 Architecture et environnement
03 Données, contrôles de qualité et règles de calcul
04 Stockage et validation dans HDFS
05 Traitements MapReduce et résultats
06 Configuration et expérimentation : Reducers, Mappers, YARN
07 Tests, vérifications et difficultés
08 Tableau de bord Python
09 Image Docker construite et publiée
10 Réponses aux 12 questions de compréhension
11 Répartition des tâches et soutenance

A Annexe A : guide de reproduction pas à pas (étapes 0 à 10)
B Annexe B : code source commenté
C Annexe C : preuves d'exécution

> **Démarche suivie**
> **Contrôles du CSV → dépôt HDFS → job détaillé (1 lecture du CSV) → 4 jobs chaînés → expériences Reducers, Mappers, YARN → tests → tableau de bord → image Docker publiée**

Le corps du rapport (sections 01 à 11) répond à la grille d'évaluation. Les annexes donnent tout ce qu'il faut pour reconstruire l'environnement chez soi : chaque commande expliquée (annexe A), le code complet commenté (annexe B) et les preuves d'exécution (annexe C).

SECTION 01

Présentation et correspondance avec la grille

En bref Une enseigne fictive de 8 magasins dans 6 villes canadiennes veut comparer ses magasins, suivre son chiffre d'affaires et repérer ses meilleurs produits. Nous avons déposé ses 24 000 ventes de 2025 dans HDFS, calculé les indicateurs avec des Mappers et un Reducer Python exécutés par Hadoop Streaming sur un cluster de 3 nœuds, puis affiché les résultats dans un tableau de bord Streamlit publié en image Docker. Le CA net 2025, **1 946 613,14 $**, est retrouvé au cent près par 5 jobs, 3 configurations de Reducers, un test local et un test calculé à la main.

Élément	Valeur
Données	`ventes_multimagasin_2025.csv` : 24 000 ventes du 1er janvier au 31 décembre 2025, 8 magasins, 6 villes, 20 produits, 4 catégories
Cluster	1 maître et 2 workers, conteneurs Docker Ubuntu 22.04.1 LTS, Hadoop 3.3.6 (HDFS et YARN), Python 3.10
Traitements	5 programmes Python (2 Mappers de production, 1 Mapper de contrôle, 1 Reducer, 1 script de contrôle qualité) ; 13 jobs Hadoop Streaming exécutés, tous `SUCCEEDED`
Application	Streamlit 1.66.1, Plotly 7.1.0, pandas 3.1.0 : 4 filtres, 4 analyses obligatoires, tableau croisé dynamique, onglet de provenance
Distribution	Image `docker.io/yacinelezoul/dashboard-ventes:1.0`, digest `sha256:412f047f…71caa7` ; dépôt GitHub public

1.1 Livrables demandés et où les trouver

Livrable demandé	Où il se trouve
1. Rapport PDF	`rapport.pdf` : rapport d'évaluation de 10 pages, organisé selon la grille ; ce document, `rapport_detaille.pdf` : version complète avec annexes
2. Code source	`sources/` (Mappers, Reducer, contrôle qualité), `application_python/` (tableau de bord, `requirements.txt`, `Dockerfile`), `tests/`, `scripts_execution/commandes_execution.md` (toutes les commandes commentées, dans l'ordre) et `scripts_execution/journaux/` (sorties des commandes), `README.md` ; commenté en annexe B
3. Image Docker publiée	Section 9 et `preuve_docker.txt` (registre, image:tag, digest, sorties `docker pull` et `docker run`)
4. PowerPoint	`presentation.pptx` : 12 diapositives, 2 par membre
5. Présentation orale	Répartition et déroulé de la démonstration en section 11 ; réponses préparées en section 10

1.2 Correspondance avec la grille d'évaluation (35 points)

Critère (points)	Ce qui est attendu	Où le rapport y répond
Compréhension des données, contrôles et dépôt HDFS (4)	Structure, contrôles de qualité, règles de calcul ; cluster vérifié ; dossiers HDFS ; fichier disponible, lisible et cohérent	3. Données ; 4. HDFS ; annexe A, étapes 2 à 4
Traitements Hadoop Streaming / MapReduce Python et exactitude (9)	Paires clé-valeur justifiées ; jobs réellement exécutés par Hadoop Streaming ; sorties HDFS distinctes ; résultats exacts	5. MapReduce ; annexe A, étape 5 ; annexe B
Justification Mappers, Reducers, partitions, YARN et tests (4)	2 Reducers et plus ; Mappers et InputSplits ; mémoire et vCores ; Shuffle ; petits cas de test	6. Expérimentation ; 7. Tests
Tableau de bord Python (5)	4 indicateurs ; au moins 2 filtres ; unités, période, provenance ; pas de lecture du CSV	8. Tableau de bord
Image Docker (4)	Dockerfile, dépendances, construction, test local, publication, mode d'emploi, preuves	9. Image Docker ; `preuve_docker.txt`
Rapport : clarté, architecture, preuves, traçabilité (3)	Architecture, choix techniques, captures, contributions	2. Architecture ; annexes A et C ; 11. Répartition
PowerPoint et démonstration (3)	8 à 12 diapositives, démonstration HDFS → tableau de bord, preuve de publication	11. Soutenance ; `presentation.pptx`
Participation et réponses aux questions (3)	Chacun explique sa partie et répond aux questions	10. Réponses aux 12 questions ; 11. Répartition

SECTION 02

Architecture et environnement

En bref Le CSV n'est lu qu'une fois, par un job qui produit une table détaillée (`ca_detail`). Les 4 indicateurs en sont tirés par chaînage de jobs, et le tableau de bord, empaqueté dans une image Docker, lit uniquement ces sorties MapReduce.

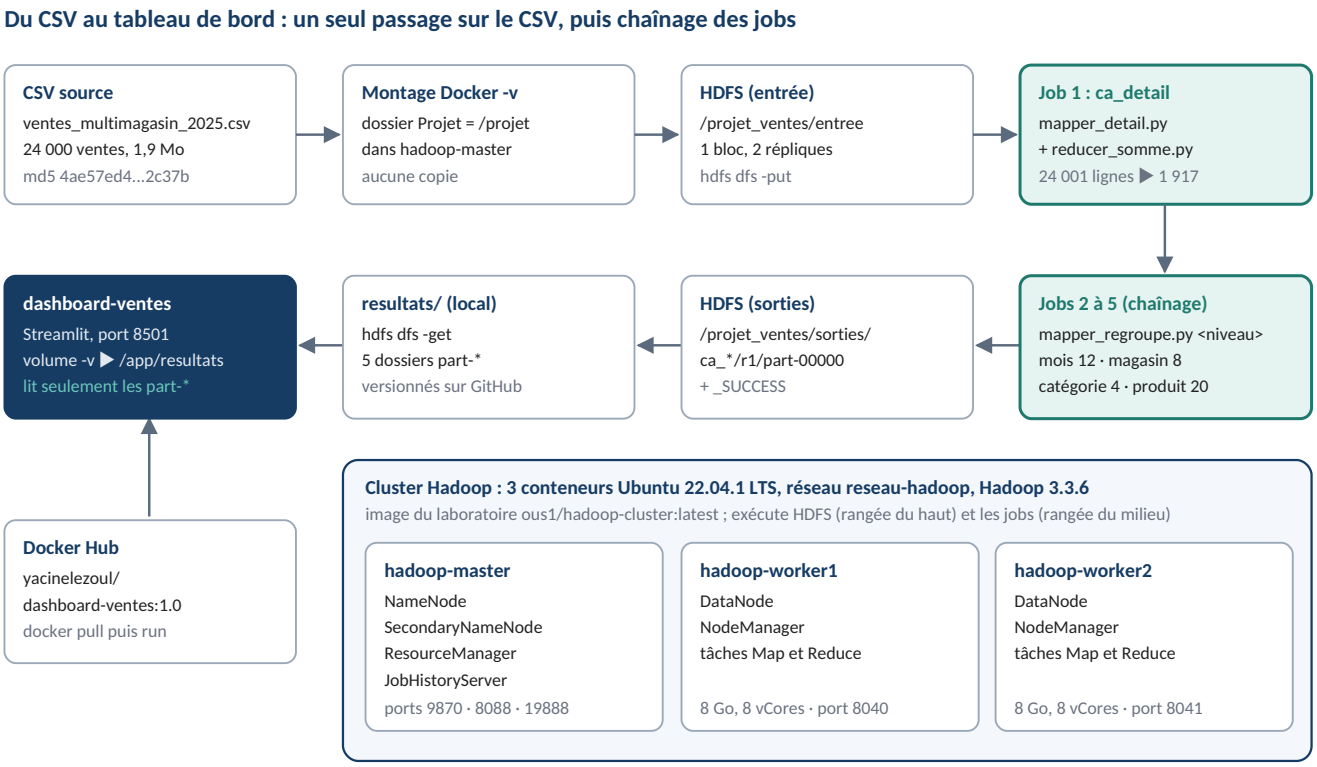


Figure 1. *Architecture de la chaîne : stockage HDFS, job détaillé, jobs chaînés, récupération des sorties et application conteneurisée*

2.1 Environnement imposé et environnement utilisé

Composant	Exigence de l'énoncé	Réalisation
Système	Ubuntu 22.04	Les 3 nœuds Hadoop sont des conteneurs **Ubuntu 22.04.1 LTS** (preuve : `cat /etc/os-release`). Docker est hébergé sur un poste Windows 11 avec Docker Desktop, qui exécute les conteneurs dans une machine virtuelle Linux.
Plateforme Big Data	Hadoop 3.3.6, HDFS et YARN	Hadoop 3.3.6 (`/usr/local/hadoop`), Java 1.8.0_402, image du laboratoire `ous1/hadoop-cluster:latest`
Cluster	1 maître, au moins 2 workers	`hadoop-master` , `hadoop-worker1` , `hadoop-worker2` : 2 DataNodes et 2 NodeManagers actifs

Programmation	Python 3 via Hadoop Streaming	Python 3.10.12 sur chaque nœud, `mapred streaming` (jar `hadoop-streaming-3.3.6.jar`)
Visualisation	Application Python	Streamlit 1.66.1, Plotly 7.1.0, pandas 3.1.0 (Python 3.14)
Conteneurisation	Image applicative publiée	Image `python:3.14-slim` publiée sur Docker Hub

Toutes les commandes Hadoop, HDFS et MapReduce de ce rapport sont des commandes Ubuntu, exécutées dans `hadoop-master`. Seules les commandes de pilotage de Docker (`docker run`, `docker exec`…) sont tapées dans PowerShell ; le README et l'annexe A donnent leur équivalent bash pour un hôte Ubuntu.

2.2 Choix d'architecture : un job détaillé, puis le chaînage

L'énoncé laisse libre d'utiliser « un ou plusieurs jobs MapReduce » si l'architecture est argumentée. Nous avons retenu le chaînage :

- **Une seule lecture du CSV.** Le job 1 émet une clé qui contient toutes les dimensions (`mois|magasin|ville|categorie|id_produit|nom_produit`) et réduit 24 001 lignes à 1 917. Les jobs 2 à 5 relisent ces 1 917 lignes : 106 622 octets lus au lieu de 1 990 145, soit 12 fois moins de lignes et 19 fois moins d'octets.
- **Une source unique pour les filtres.** Une clé fine permet toutes les vues plus grossières : le tableau de bord croise ville, magasin, catégorie, produit et mois à partir de `ca_detail`, sans relire le CSV. L'inverse est impossible : un total par ville ne peut plus être ventilé par mois.
- **Un contrôle indépendant.** Le premier job, calculé directement depuis le CSV (`mapper_mois.py`), est conservé : son résultat a la même empreinte md5 que le CA mensuel obtenu par chaînage.
- **Le mois comme grain temporel.** Avec 7,7 ventes par jour et par magasin, le CA journalier varie de 60 % autour de sa moyenne (coefficient de variation 0,6) et ne montre aucun effet du jour de la semaine (±5 %), alors que la tendance mensuelle est nette (+86 % de janvier à décembre).

SECTION 03

Données, contrôles de qualité et règles de calcul

En bref Le script `sources/controle_qualite.py` a vérifié le CSV avant tout traitement : structure conforme, aucune valeur manquante, aucun doublon d'identifiant, référentiels cohérents. Une seule décision a été nécessaire : garder 334 lignes identiques à une autre à l'identifiant près. Le script ne calcule aucun indicateur ; il contrôle seulement la source.

Contrôle	Résultat (`resultats/controle_qualite.txt`)
Structure et encodage	En-tête identique au dictionnaire de l'énoncé (12 colonnes) ; 24 000 lignes de données ; 0 ligne sans 12 colonnes ; lecture UTF-8 sans erreur ; séparateur virgule, aucun guillemet
Valeurs manquantes	0 dans chacune des 12 colonnes
Doublons	0 `id_vente` en double ; 0 ligne entière en double ; **334 lignes identiques hors `id_vente`**
Dates	0 date invalide ; période du 2025-01-01 au 2025-12-31
Référentiels	8 magasins, 6 villes, chaque magasin a une seule ville ; 20 produits, chacun avec un seul nom, une seule catégorie et un seul prix ; 4 catégories
Valeurs numériques	`quantite` entière de 1 à 5 ; `prix_unitaire_cad` de 6,50 $ à 219,00 $; `remise_pct` dans {0, 5, 10, 15, 20} ; aucune valeur invalide
Codes et répartitions	`statut` : VALIDEE 22 576, ANNULEE 1 424 ; `mode_paiement` : CARTE 17 246, ESPECES 5 304, VIREMENT 1 450 ; de 2 359 (M02) à 4 646 (M05) ventes par magasin

3.1 Règles de calcul

- **Montant net** d'une vente VALIDEE = `quantite × prix_unitaire_cad × (1 − remise_pct / 100)`, en dollars canadiens hors taxes.
- **Arrondi au cent par vente, avant toute somme**, avec la règle « 0,5 arrondi vers le haut » (`Decimal` et `ROUND_HALF_UP`). Les montants circulent ensuite en **cents entiers** : additionner des entiers est toujours exact.
- **Exclusions** : les ventes ANNULEE et les lignes mal formées sont ignorées par le Mapper ; elles n'émettent aucune paire.

Pourquoi `Decimal` et non `float`. 1 749 ventes (7,7 % des ventes validées) tombent pile sur un demi-cent. En flottant, `round(6.50 * 0.95, 2)` donne **6.17** au lieu de 6,18, car 6,175 est stocké sous la forme 6,17499… ; et l'arrondi « au pair » de Python fait varier le total annuel de 8,51 $. Avec `Decimal`, le résultat est exact et conforme à la règle de l'énoncé.

3.2 Décision sur les 334 lignes identiques hors identifiant

334 lignes (1,4 % du fichier) reprennent la date, le magasin, le produit, la quantité, la remise, le statut et le mode de paiement d'une autre ligne, avec un `id_vente` différent. **Nous les avons conservées** : chaque identifiant est unique, ce qui indique que la caisse les a enregistrées comme des transactions distinctes ; deux achats identiques le même

jour dans le même magasin sont plausibles pour des produits courants ; rien dans les données ne prouve une double saisie, et les supprimer réduirait le chiffre d'affaires sans justification.

SECTION 04

Stockage et validation dans HDFS

En bref Le cluster a été vérifié avant tout traitement (2 DataNodes, 2 NodeManagers). Le CSV a été déposé deux fois dans HDFS : avec des blocs de 128 Mo (1 bloc) et de 1 Mo (2 blocs, pour l'expérience sur les Mappers). Nombre de lignes, empreinte md5 et emplacement des répliques prouvent que la copie HDFS est identique au fichier local.

Organisation HDFS

```
/projet_ventes/
├── entree/            # CSV, blocs de 128 Mo
├── entree_blocs_1m/   # même CSV, blocs de 1 Mo
├── tests/             # echantillon_test.csv
└── sorties/
    ├── ca_detail/r1        # job 1
    ├── ca_mois/r1 r2 r4    # 1, 2, 4 Reducers
    ├── ca_magasin/r1
    ├── ca_categorie/r1
    ├── ca_produit/r1
    ├── controle_ca_mensuel_direct/r1
    ├── exp_mappers/maps1 maps4 blocs1m_maps1
    └── tests/ca_detail ca_mois
```

Vérification	Résultat
`hdfs dfsadmin -report`	**Live datanodes (2)**
`yarn node -list`	**2 nœuds RUNNING**
Lignes : local / HDFS	24 001 / 24 001
md5 : local / HDFS	`4ae57ed4…2c37b` identiques
`fsck` entree/	1 bloc de 1 985 795 o, `Live_repl=2`, HEALTHY
`fsck` entree_blocs_1m/	2 blocs : 1 048 576 + 937 219 o
Répliques	172.18.0.2 (worker1) et 172.18.0.3 (worker2)

Avec 2 DataNodes et une réplication de 2, chaque worker détient une copie complète de chaque bloc : la perte d'un worker ne rend aucune donnée inaccessible. Le fichier local n'est jamais copié sur Linux : un montage Docker (`-v`) rend le dossier du projet visible dans le maître sous `/projet`, d'où `hdfs dfs -put` l'envoie dans HDFS.

SECTION 05

Traitements MapReduce et résultats

En bref Deux Mappers et un seul Reducer suffisent : le Mapper choisit la clé, le Reducer additionne les cents d'une même clé, quelle que soit sa signification. Tous les jobs ont tourné par Hadoop Streaming sur le cluster, avec une sortie HDFS distincte par job et par configuration.

5.1 Paires clé-valeur

La clé répond à la question « **par quoi** regrouper ? » et la valeur à « **quoi** additionner ? ». La valeur est toujours le montant net en cents. L'`id_vente` ne peut pas servir de clé : unique, il produirait 24 000 groupes d'une seule vente et aucun regroupement. Les morceaux d'une clé composée sont séparés par `|`, car la virgule est le séparateur du CSV et la tabulation celui de Hadoop Streaming entre clé et valeur.

Job	Mapper	Clé émise	Exemple de sortie (cents)	Lignes
1 ca_detail	`mapper_detail.py`	mois|magasin|ville|categorie|id_produit|nom_produit	`2025-01|M01|Ottawa|Bureau|P006|Cahier A5 → 12709`	1 917
2 ca_mois	`mapper_regroupe.py` + niveau	mois	`2025-01 → 12331669`	12
3 ca_magasin		magasin|ville	`M05|Toronto → 37726841`	8
4 ca_categorie		categorie	`Electronique → 79637763`	4
5 ca_produit		id_produit|nom|categorie	`P004|Ecran 24 pouces|Electronique → 26667630`	20
Contrôle direct	`mapper_mois.py`	mois (depuis le CSV)	même résultat que le job 2, md5 `db30e50f…a634`	12

5.2 Exécution par Hadoop Streaming

Hadoop Streaming permet d'écrire Mapper et Reducer en Python : il envoie les lignes de chaque InputSplit sur l'entrée standard du Mapper, lit ses paires `clé<TAB>valeur` sur sa sortie standard, les trie et les regroupe par clé (Shuffle & Sort), puis les envoie au Reducer. L'option `-files` copie les scripts sur chaque worker ; `-input` et `-output` désignent des chemins HDFS. Le job 1 :

Commande (dans hadoop-master)

```
mapred streaming \
  -D mapreduce.job.name="ca_detail_r1" -D mapreduce.job.reduces=1 \
  -files /projet/sources/mapper_detail.py,/projet/sources/reducer_somme.py \
  -mapper "python3 mapper_detail.py" -reducer "python3 reducer_somme.py" \
  -input /projet_ventes/entree/ventes_multimagasin_2025.csv \
  -output /projet_ventes/sorties/ca_detail/r1
```

Compteurs du job 1 : `Map input records=24001` (en-tête compris), `Map output records=22576` (une paire par vente validée ; les 1 424 annulées n'en produisent pas), `Reduce input groups=1917`, `Reduce output records=1917`. Les

jobs 2 à 5 prennent pour entrée le dossier `/projet_ventes/sorties/ca_detail/r1` et lancent `python3 mapper_regroupe.py <niveau>` ; ils lisent 1 917 lignes chacun. Le code complet est en annexe B.

5.3 Résultats des 4 analyses obligatoires

Mois 2025	CA net ($)
Janvier	123 316,69
Février	129 106,01
Mars	143 888,51
Avril	145 830,86
Mai	154 270,70
Juin	140 922,88
Juillet	166 776,54
Août	156 007,11
Septembre	159 640,31
Octobre	186 641,81
Novembre	211 219,00
Décembre	228 992,72
Total	**1 946 613,14**

Magasin	Ville	CA net ($)
M05	Toronto	377 268,41
M01	Ottawa	276 695,18
M03	Montréal	244 011,03
M06	Québec	222 651,74
M04	Montréal	217 282,75
M08	Kingston	214 109,03
M07	Gatineau	206 204,68
M02	Ottawa	188 390,32

Catégorie	CA net ($)	Part
Electronique	796 377,63	40,9 %
Maison	627 786,89	32,3 %
Sport	320 095,04	16,4 %
Bureau	202 353,58	10,4 %

Ville	CA net ($)
Ottawa (2 mag.)	465 085,50
Montréal (2 mag.)	461 293,78
Toronto	377 268,41
Québec	222 651,74
Kingston	214 109,03
Gatineau	206 204,68

Top 5 produits	CA net ($)
1. Ecran 24 pouces	266 676,30
2. Aspirateur compact	225 395,30
3. Casque audio	193 918,15
4. Cafetiere	151 896,30
5. Webcam HD	142 774,10

Lecture. Le CA progresse de 86 % entre janvier et décembre, avec une forte accélération au dernier trimestre. Toronto (un seul magasin, M05) est le premier magasin, mais Ottawa et Montréal, avec deux magasins chacune, sont les premières villes. L'électronique pèse 41 % du CA, et 3 des 5 meilleurs produits en relèvent. Le Top 5 est obtenu en triant les 20 lignes du job `ca_produit` : un tri de résultats déjà agrégés, et non un recalcul.

SECTION 06

Configuration et expérimentation : Reducers, Mappers, YARN

En bref 8 exécutions comparées : 1, 2 et 4 Reducers sur `ca_mois` ; 1, 2 et 4 Mappers sur le CSV, et 1 Mapper demandé sur le fichier à blocs de 1 Mo. Le total reste 194 661 314 cents dans tous les cas. Sur ce petit volume, ajouter des tâches coûte plus en démarrage de conteneurs que ça ne rapporte.

6.1 Nombre de Reducers et partitions de sortie

Mesure (journal 05)	1 Red.	2 Red.	4 Red.
Fichiers `part-*`	1	2	4
Mois par fichier	12	7 / 5	4/2/3/3
Durée du job	23 s	27 s	34 s
Temps cumulé des Reducers	6,2 s	15,9 s	74,4 s
Mémoire physique	0,95 Go	1,23 Go	1,73 Go
`Shuffled Maps`	2	4	8
Total (cents)	194 661 314 dans les 3 cas		

Fichier (4 Reducers)	Mois
`part-00000`	01, 05, 09, 12
`part-00001`	02, 06
`part-00002`	03, 07, 10
`part-00003`	04, 08, 11

La partition d'une clé est `hash(clé) mod R`. Nous l'avions calculée à l'avance pour les 12 mois : la sortie réelle correspond exactement.

Chaque mois apparaît dans un seul fichier : une même clé a toujours le même hash, donc le même Reducer, qui reçoit toutes ses valeurs. Deux clés différentes peuvent partager un Reducer (4 mois dans `part-00000`) et la répartition n'est pas équilibrée. Chaque Reducer est un conteneur YARN d'au moins 1 Go à démarrer : pour 1 917 lignes, ce coût dépasse le gain du parallélisme. Le compteur `Killed reduce tasks=1` du job à 2 Reducers n'est pas une erreur : c'est l'exécution spéculative, qui lance une copie de secours d'une tâche lente et garde la première terminée.

6.2 Nombre de Mappers et InputSplits

Configuration (journal 06)	Mappers demandés	Taille de bloc	InputSplits = Mappers	Octets lus	Durée
maps1	1	128 Mo	**1**	1 985 795	14 s
Défaut (job ca_detail)	2	128 Mo	**2**	1 989 891	39 s
maps4	4	128 Mo	**4**	1 998 083	21 s
blocs1m_maps1	1	1 Mo	**2**	1 989 891	17 s

Hadoop Streaming demande 2 Mappers par défaut (`mapreduce.job.maps=2`) : un fichier d'un seul bloc a donc donné 2 Mappers. La règle observée est : **taille d'un InputSplit = minimum entre (taille du fichier ÷ Mappers demandés) et taille d'un bloc**. Sur le fichier à blocs de 1 Mo, un InputSplit ne peut pas dépasser 1 Mo : 2 Mappers sont lancés même si on en demande 1. Les octets lus montrent le mécanisme des fins de ligne : chaque coupure entre deux InputSplits ajoute 4 096 octets, car le Mapper de gauche lit au-delà de sa limite pour terminer la ligne coupée (la vente V012677 est coupée entre « V0126 » et « 77 » à l'octet 1 048 576). Le nombre de DataNodes et de répliques n'intervient pas.

6.3 Ressources YARN

Paramètre ou mesure	Valeur	Effet sur l'exécution
`yarn.nodemanager.resource.memory-mb` / `cpu-vcores`	8 192 Mo / 8 par worker	Capacité offerte par chaque NodeManager ; total du cluster 16 Go et 16 vCores (journal 07)
`yarn.scheduler.minimum-allocation-mb` / vCores	1 024 Mo / 1	Taille minimale d'un conteneur : un worker accueille au plus 8 tâches simultanées
`yarn.scheduler.maximum-allocation-mb` / vCores	8 192 Mo / 4	Taille maximale qu'une tâche peut demander
Mémoire et vCores par tâche (mesurés)	1 024 Mo / 1	`megabyte-milliseconds` ÷ `milliseconds` = 4 263 936 ÷ 4 164 = 1 024 ; `vcore-milliseconds` = temps (règlent `mapreduce.map.memory.mb` et `mapreduce.reduce.memory.mb`)
Bilan du job maps4 (`yarn application -status`)	130 124 Mo·s, 90 vCore·s	Inclut l'ApplicationMaster (lancé sur hadoop-worker2) et toute la vie des conteneurs ; supérieur aux 51 000 Mo·s des seules tâches
Mémoire réellement utilisée par tâche	≈ 357 Mo	`Peak Map Physical memory` : bien en dessous des 1 024 Mo réservés

Limite constatée. YARN déclare 16 Go et 16 vCores alors que la machine hôte n'a que 7,47 Go et 4 processeurs : YARN fait confiance à sa configuration et ne vérifie pas le matériel. Cela fonctionne ici parce que chaque tâche n'utilise qu'environ 357 Mo, mais sur un gros volume cette surréservation ferait manquer de mémoire aux conteneurs.

SECTION 07

Tests, vérifications et difficultés

En bref Un mini-CSV de 7 ventes piégées, calculé à la main, est retrouvé exactement en local, sur Hadoop, et à travers le chaînage. Six autres contrôles croisés confirment l'exactitude des résultats sans se fier à l'apparence des graphiques.

7.1 Cas de test calculé à la main (`tests/` , journal 10)

Ligne	Piège testé	Calcul à la main	Attendu
T01	Vente simple	1 × 219,00 × 1,00	219,00 $
T02	Remise 15 %, demi-cent	1 × 8,90 × 0,85 = 7,565	7,57 $
T03	Même mois, magasin, produit que T02	2 × 8,90 × 0,85 = 15,13	additionnée à T02 : 22,70 $
T04	Remise 20 %, quantité 3	3 × 219,00 × 0,80	525,60 $
T05	Vente ANNULEE		ignorée
T06	Ligne de 11 colonnes		ignorée, sans plantage
T07	Demi-cent piégé en flottant	1 × 6,50 × 0,95 = 6,175	6,18 $ (et non 6,17)

Résultats : TEST LOCAL DETAIL : REUSSI TEST HADOOP DETAIL : REUSSI TEST HADOOP CHAINAGE : REUSSI. Compteurs Hadoop : `Map input records=8`, `Map output records=5` (T05 et T06 écartées), `Reduce output records=4` (T02 et T03 additionnées) ; total 77 348 cents. La comparaison est faite par `diff` avec les fichiers attendus, triés octet par octet (`LC_ALL=C`) comme le fait Hadoop.

7.2 Contrôles de cohérence

Contrôle	Résultat
Chaîne locale (`mapper | sort | reducer`) comparée à Hadoop	**12 mois identiques, 194 661 314 cents**
Chaînage comparé au calcul direct depuis le CSV	**md5 identiques : db30e50fbe9042172383b551c5a4a634**
1, 2 et 4 Reducers (somme de tous les `part-*`)	**194 661 314 cents dans les 3 cas**
1, 2 et 4 Mappers	**`Reduce output records=1917` dans tous les cas**
5 jobs de production	**même total ; affiché dans l'onglet Provenance du tableau de bord**
Compteurs	**22 576 validées + 1 424 annulées = 24 000 ; 24 001 lignes lues avec l'en-tête**

7.3 Difficultés, anomalies et limites

Difficulté	Symptôme	Démarche de résolution

Arrondi au demi-cent	Premier total de contrôle calculé en flottant : 194 660 289 cents, faux de 10,25 $	Analyse des 1 749 demi-cents ; `Decimal` + `ROUND_HALF_UP` ; montants en cents ; total corrigé : 194 661 314
Nombre de Mappers inattendu	2 Mappers pour un fichier d'un seul bloc	Lecture des compteurs ; Hadoop Streaming demande 2 Mappers par défaut ; expérience 6.2 avec `mapreduce.job.maps`
Anciennes installations	2 clusters (5 workers) et des conteneurs orphelins sur la machine	Inventaire, puis suppression (`docker compose down -v` , `docker rm -f`) ; cluster recréé avec les noms attendus par l'image
Deux terminaux mélangés	Commande PowerShell tapée dans bash, `<IMAGE>` refusé par PowerShell	Repère systématique de l'invite : `PS C:\>` pour Docker, `root@hadoop-master` pour Hadoop
Fins de ligne Windows	Git convertit `\n` en `\r\n` , ce qui changerait le md5 du CSV	`.gitattributes` : `*.py eol=lf` , `*.csv -text`
Affichage du tableau de bord	« $…$ » lu comme une formule mathématique ; courbes en zigzag	Échappement `\$` (`texte_md`) ; tri des courbes par mois
Connexion Docker Hub	Champ de mot de passe invisible, collage impossible	Connexion par le navigateur (`docker login` sans `-u`) ; aucun jeton écrit dans un fichier
Limites	Volume faible (le démarrage des conteneurs domine les temps mesurés, une seule exécution par configuration) ; surréservation YARN (6.3) ; hôte Windows pour Docker, cluster sous Ubuntu 22.04	

SECTION 08

Tableau de bord Python

En bref Une application Streamlit en 3 onglets lit les 1 917 lignes de `ca_detail` et propose 4 filtres qui agissent sur toute la page. Elle affiche les 4 analyses obligatoires, un tableau croisé dynamique avec sous-totaux et un onglet qui prouve la provenance des données. Elle ne lit jamais le CSV.



Figure 2. *Filtre Ottawa + Gatineau : indicateurs recalculés (671 290 $, 34,5 % du CA annuel, +88 %) et courbe mensuelle de la sélection*

Fonction	Contenu
Filtres (4)	Ville, magasin (limité aux villes choisies), catégorie, période (curseur de janvier à décembre) ; un filtre vide signifie « toutes les valeurs »
Indicateurs (4)	CA net de la sélection et part du CA annuel ; évolution entre le premier et le dernier mois ; meilleur mois ; meilleur magasin
Analyses obligatoires	1. CA mensuel (une courbe au total, par ville, par magasin ou par catégorie) ; 2. CA par magasin regroupé par ville, et carte proportionnelle ville → magasin ; 3. CA par catégorie (anneau) et poids des catégories dans chaque ville ; 4. Top produits (5 à 20, curseur)
Tableau croisé	Niveaux en lignes au choix et dans l'ordre voulu (ville, magasin, catégorie, produit), mois en colonnes, sous-totaux colorés, total général ; un accordéon par ville avec son CA et sa part
Lisibilité	Une couleur fixe par ville et par catégorie dans tous les graphiques ; montants au format canadien (`1 946 613,14 $`) ; période, unité (CA net en dollars canadiens hors taxes, ventes VALIDEE) et source indiquées en haut de page
Provenance et contrôles	Dossier lu, liste des fichiers `part-*` avec leur nombre de lignes, totaux des 5 jobs comparés, règles de calcul

Preuve que l'application exploite les sorties MapReduce. Un seul module accède aux données, `lecture_resultats.py`, et il ne lit que les fichiers `part-*` du dossier `resultats/` (ou de `/app/resultats` dans le conteneur). Le code ne contient aucun chemin vers le CSV. L'onglet Provenance affiche les fichiers lus (12, 8, 4, 20 et 1 917 lignes) et les totaux des 5 jobs. Enfin, l'image Docker ne contient aucune donnée : sans le volume des résultats MapReduce, elle n'a rien à afficher. L'application ne fait que filtrer et regrouper des lignes déjà agrégées par MapReduce.

SECTION 09

Image Docker construite et publiée

En bref L'image `yacinelezoul/dashboard-ventes:1.0` a été construite, testée, publiée sur Docker Hub, supprimée de la machine, puis téléchargée et relancée depuis le registre : le digest téléchargé est identique au digest publié.

Élément	Valeur
Registre	Docker Hub (public) : https://hub.docker.com/r/yacinelezoul/dashboard-ventes
Image:tag	`yacinelezoul/dashboard-ventes:1.0`
Digest	`sha256:412f047f4e70…5971caa7`
Taille	190 Mo compressée (Docker Hub)
Base	`python:3.14-slim`
Dépendances	streamlit 1.66.1, plotly 7.1.0, pandas 3.1.0
Données	aucune : volume `-v …/resultats:/app/resultats:ro`

Récupérer et lancer (Linux ; Windows : ${PWD}\resultats)

```
git clone https://github.com/lezoul/hadoop-mapreduce-ventes-multimagasins.git
cd hadoop-mapreduce-ventes-multimagasins
docker pull yacinelezoul/dashboard-ventes:1.0
docker run -d --name dashboard-ventes -p 8501:8501 \
  -v "$(pwd)/resultats:/app/resultats:ro" \
  yacinelezoul/dashboard-ventes:1.0
# http://localhost:8501
```

Construction. Le Dockerfile copie `requirements.txt` et installe les bibliothèques avant de copier le code : Docker réutilise cette couche tant que les dépendances ne changent pas. Le premier `docker build` a pris 172 s, dont 102 s d'installation ; le second, 1,4 s (toutes les étapes `CACHED`). La variable `DOSSIER_RESULTATS=/app/resultats` indique à l'application où lire les résultats ; `--server.address=0.0.0.0` la rend accessible hors du conteneur.

Vérification de la publication (journal 11 et `preuve_docker.txt`) : `docker push` (digest `412f047f…`) → `docker rm -f` et `docker rmi` → `docker pull` (`Downloaded newer image`, même digest) → `docker run` → `docker ps` (`0.0.0.0:8501->8501/tcp`) ; l'onglet Provenance affiche alors `/app/resultats`. **Sécurité** : la connexion à Docker Hub s'est faite par le navigateur ; aucun mot de passe ni jeton n'est écrit dans le code, l'image ou le dépôt.

SECTION 10

Réponses aux 12 questions de compréhension

1. Pourquoi stocker les transactions dans HDFS plutôt que de conserver uniquement le CSV sur la machine locale ?

HDFS découpe le fichier en blocs répliqués sur plusieurs machines : il survit à la perte d'un nœud (nos 2 répliques sur worker1 et worker2) et permet de traiter chaque bloc là où il est stocké, en parallèle (`Data-local map tasks=2`). Un CSV local dépend d'un seul disque et d'une seule machine, et ne passe pas à l'échelle quand le volume dépasse sa capacité.

2. Quelle différence entre un bloc HDFS, un InputSplit et une tâche Mapper ?

Le bloc est un découpage **physique** du stockage, au nombre d'octets (128 Mo par défaut ; 1 Mo dans notre expérience). L'InputSplit est un découpage **logique** de la lecture, ajusté aux fins de ligne : la vente V012677, coupée entre deux blocs, est lue entièrement par le premier Mapper. La tâche Mapper est le programme qui traite un InputSplit : il y a un Mapper par InputSplit.

3. Le nombre de DataNodes ou de répliques détermine-t-il automatiquement le nombre de Mappers ?

Non. Il dépend du nombre d'InputSplits, donc de la taille du fichier, de la taille des blocs et des paramètres de découpage. Avec les mêmes 2 DataNodes et 2 répliques, nous avons obtenu 1, 2 ou 4 Mappers selon `mapreduce.job.maps`, et 2 Mappers en en demandant 1 sur le fichier à blocs de 1 Mo. Les répliques ne sont pas lues deux fois : seule la copie la plus proche est lue.

4. Quel est le rôle de Hadoop Streaming lorsque Mapper et Reducer sont écrits en Python ?

Il fait le lien entre le framework Java de Hadoop et des programmes quelconques : il distribue les scripts (`-files`), envoie chaque ligne de l'InputSplit sur l'entrée standard du Mapper, lit sa sortie standard en coupant clé et valeur sur la première tabulation, fait le Shuffle & Sort, puis envoie au Reducer les lignes triées par clé.

5. Comment le choix d'une clé influence-t-il le regroupement par magasin, par mois ou par catégorie ?

Toutes les valeurs d'une même clé sont additionnées ensemble : la clé est le critère de regroupement. `2025-03` donne le CA de mars, `M05|Toronto` celui d'un magasin, `Sport` celui d'une catégorie. Une clé composée au grain le plus fin (`ca_detail`, 1 917 groupes) permet de reconstituer tous les regroupements plus grossiers ; une clé unique comme `id_vente` n'en produirait aucun.

6. Que réalise la phase Shuffle / Sort et comment les valeurs d'une même clé arrivent-elles au bon Reducer ?

Chaque paire émise par un Mapper reçoit une partition, `hash(clé) mod nombre_de_Reducers`. Le Shuffle copie vers chaque Reducer toutes les paires de sa partition, venant de tous les Mappers (`Shuffled Maps` = Mappers × Reducers : 2, 4, 8), puis le Sort les trie par clé. Le Reducer reçoit ainsi toutes les valeurs d'une clé à la suite et peut les additionner en une seule passe ; en local, la commande `sort` jouait ce rôle.

7. Pourquoi deux clés différentes peuvent-elles aller au même Reducer, alors qu'une clé identique reste dans la même partition ?

Le calcul de partition est déterministe : une clé donne toujours le même hash, donc la même partition. Mais il y a moins de Reducers que de clés (4 Reducers pour 12 mois) : plusieurs clés tombent forcément sur le même reste. Nos 4 fichiers contiennent 4, 2, 3 et 3 mois, et chaque mois n'apparaît qu'une fois.

8. Que change le passage de 1 à 2, puis à 4 Reducers ?

On obtient 1, 2 puis 4 fichiers `part-*`, chacun avec une partie des clés, et un Shuffle de 2, 4 puis 8 transferts ; le total ne change pas. Plus de Reducers parallélise la réduction sur un gros volume ; sur notre petit volume, la durée est passée de 23 à 34 s et la mémoire de 0,95 à 1,73 Go, car chaque Reducer est un conteneur à démarrer.

9. Quels paramètres YARN relatifs à la mémoire et aux vCores ont un impact sur l'exécution ?

`yarn.nodemanager.resource.memory-mb` et `yarn.nodemanager.resource.cpu-vcores` (8 192 Mo et 8 vCores par worker) fixent la capacité de chaque nœud ; `yarn.scheduler.minimum-allocation-mb` et `maximum-allocation-mb` / `-vcores` (1 024 Mo et 1 vCore, 8 192 Mo et 4 vCores) bornent chaque conteneur ; `mapreduce.map.memory.mb`, `mapreduce.reduce.memory.mb` et `yarn.app.mapreduce.am.resource.mb` fixent ce que demandent les tâches et l'ApplicationMaster (1 024 Mo et 1 vCore mesurés par tâche). Ensemble, ils déterminent combien de tâches tournent en même temps sur chaque worker (8 ici).

10. Comment évitez-vous de compter des transactions annulées dans le chiffre d'affaires ?

Le Mapper teste `champs[10] != "VALIDEE"` et passe à la ligne suivante sans rien émettre : une vente annulée n'atteint jamais le Reducer. Preuves : `Map output records=22576` pour 24 000 ventes (1 424 annulées), et la ligne T05 du cas de test, absente du résultat.

11. Comment démontrez-vous que le tableau de bord exploite les résultats MapReduce plutôt que le CSV ?

Le seul module de lecture (`lecture_resultats.py`) ne lit que des `part-*` ; aucun chemin vers le CSV n'existe dans l'application ; l'onglet Provenance liste les fichiers lus et leurs lignes ; l'image ne contient pas de données et affiche `/app/resultats`, le volume des sorties MapReduce. Les chiffres affichés sont identiques au cent près aux fichiers `part-*`.

12. Quelle différence entre une image Docker publiée et un conteneur Docker démarré ?

L'image est un modèle figé en lecture seule (système, Python, bibliothèques, code), identifié par son digest et stocké dans un registre : `yacinelezoul/dashboard-ventes:1.0`, `sha256:412f047f…`. Le conteneur est une instance en cours d'exécution de cette image, avec son propre état, ses ports (8501) et ses volumes (`/app/resultats`). Une même image peut lancer plusieurs conteneurs ; nous avons supprimé le conteneur et l'image locale, puis recréé un conteneur à partir de l'image publiée.

SECTION 11

Répartition des tâches et soutenance

En bref La réalisation technique a été menée par Yacine Lezoul ; chaque autre membre est responsable d'une vérification indépendante ou d'un livrable, et présente 2 diapositives. Chacun doit pouvoir expliquer sa partie et répondre aux 12 questions de la section 10.

Membre	Contribution	Partie présentée
Yacine Lezoul 2748526	Coordination ; création du cluster et démarrage de HDFS et YARN ; contrôle qualité ; dépôt HDFS ; conception et code MapReduce (chaînage) ; expériences Reducers, Mappers et YARN ; cas de test ; tableau de bord ; image Docker ; dépôt GitHub	Diapositives 1 et 2 : contexte, architecture
Mouncif Bouchachi 2743259	Reproduction de la chaîne sur un hôte Ubuntu 22.04 à partir du README, avec captures	Diapositives 3 et 4 : environnement, données et HDFS
Adel Benhamma 2741167	Relecture du rapport ; vérification des réponses aux 12 questions	Diapositives 5 et 6 : MapReduce, chaînage et résultats
Douyeri Lamoussa Sanogo 2731042	Mise en forme et répétition du PowerPoint ; chronométrage de la démonstration	Diapositives 7 et 8 : Reducers, Mappers, YARN
Rimy Meziani 2741840	Réexécution du cas de test (phase 8) sur sa machine à partir du README	Diapositives 9 et 10 : tests, tableau de bord
Barnabe Landry Evouna 2751210	Lancement de l'image depuis Docker Hub sur sa machine (preuve indépendante du `docker pull`)	Diapositives 11 et 12 : image Docker, démonstration et conclusion

11.1 Déroulé de la démonstration

1. **Cluster** : `docker ps`, puis `jps`, `hdfs dfsadmin -report` et `yarn node -list` dans hadoop-master.
2. **HDFS** : `hdfs dfs -ls -R /projet_ventes` et `hdfs fsck` sur le CSV (blocs et répliques).
3. **MapReduce** : lancement en direct de `ca_categorie` (le job le plus court), suivi sur http://localhost:8088, puis lecture du `part-00000`.
4. **Partitions** : contenu des 4 fichiers de `ca_mois/r4`.
5. **Tableau de bord depuis l'image publiée** : `docker pull` puis `docker run`, application de 2 filtres, onglet Provenance (`/app/resultats`).

Plan de secours : en cas de panne pendant la séance, les captures (annexe C) et les journaux (`scripts_execution/journaux/`) contiennent la trace de chaque étape.

ANNEXE A

Guide de reproduction pas à pas (étapes 0 à 10)

> **En bref** Ce guide permet à chaque membre de reconstruire tout l'environnement sur sa machine, de la création du cluster à la publication de l'image. Chaque étape indique où taper les commandes, ce qu'elles font et ce qu'il faut obtenir. Toutes ces commandes sont regroupées dans `scripts_execution/commandes_execution.md` ; les sorties réelles sont dans `scripts_execution/journaux/`.

Repères avant de commencer

Invite affichée	Où l'on est	Commandes possibles
`PS C:\Users\...>` ou `$` (hôte Ubuntu)	Machine hôte	`docker`, `git`, `python` (tableau de bord) ; chemins de la machine
`root@hadoop-master:~#`	Linux du conteneur maître	`hdfs`, `yarn`, `mapred`, `python3`, `ls`, `md5sum` ; chemins `/projet/...` (Linux) et `/projet_ventes/...` (HDFS)

Trois espaces de fichiers. (1) Le dossier du projet sur l'hôte ; (2) le même dossier vu dans hadoop-master sous `/projet`, grâce au montage `-v` (aucune copie) ; (3) HDFS, un système de fichiers distinct, réparti sur les workers, accessible seulement par `hdfs dfs`. Avec `hdfs dfs -put SOURCE DESTINATION`, la source est un chemin Linux et la destination un chemin HDFS ; `-get` fait l'inverse. Sous Windows, une ligne de commande se poursuit avec `` ` `` ; sous bash, avec `\`.

Étape 0 : récupérer le projet et l'image Hadoop

Hôte

```
git clone https://github.com/lezoul/hadoop-mapreduce-ventes-multimagasins.git
cd hadoop-mapreduce-ventes-multimagasins
docker images                    # l'image du laboratoire ous1/hadoop-cluster:latest doit être présente
```

`git clone` copie le dépôt (code, données, résultats, tests). L'image Hadoop est celle fournie au laboratoire : Ubuntu 22.04.1, Hadoop 3.3.6 dans `/usr/local/hadoop`, Java 1.8, Python 3.10.12. On peut vérifier son contenu sur un conteneur existant avec `docker exec <conteneur> bash -c 'hadoop version; ls /root; cat $HADOOP_HOME/etc/hadoop/workers'` : le fichier `workers` attend les noms `hadoop-worker1` et `hadoop-worker2`.

Étape 1 : créer le réseau et les 3 conteneurs

Hôte (bash ; sous PowerShell remplacer \ par ` et $(pwd) par ${PWD})

```
docker network create --driver bridge reseau-hadoop
docker run -itd --name hadoop-worker1 --hostname hadoop-worker1 --network reseau-hadoop -p 8040:8042 ous1/
hadoop-cluster:latest
docker run -itd --name hadoop-worker2 --hostname hadoop-worker2 --network reseau-hadoop -p 8041:8042 ous1/
hadoop-cluster:latest
docker run -itd --name hadoop-master --hostname hadoop-master --network reseau-hadoop \
  -p 9870:9870 -p 8088:8088 -p 19888:19888 \
  -v "$(pwd):/projet" ous1/hadoop-cluster:latest
docker ps
```

Élément	Rôle
`network create --driver bridge`	Réseau privé où les conteneurs se joignent par leur nom (hadoop-master écrit « hadoop-worker1 » sans connaître d'adresse IP)
`docker run -itd`	Crée et démarre un conteneur ; `-it` garde un terminal ouvert pour qu'il ne s'arrête pas, `-d` le lance en arrière-plan
`--name` / `--hostname`	Nom pour Docker / nom de machine pour Hadoop ; identiques, et conformes au fichier `workers`
`-p HÔTE:CONTENEUR`	Publie un port : 9870 interface HDFS, 8088 interface YARN, 19888 historique des jobs, 8040 et 8041 NodeManagers
`-v "$(pwd):/projet"`	Montage : le dossier du projet apparaît dans le maître sous `/projet`
`docker ps`	Attendu : les 3 conteneurs à l'état `Up`

Si d'anciens conteneurs existent, les supprimer d'abord (`docker rm -f`, `docker compose -p <projet> down -v`) sans supprimer l'image.

Étape 2 : démarrer et vérifier HDFS et YARN

Hôte, puis hadoop-master

```
docker exec -it hadoop-master bash      # ouvre un terminal Linux dans le maître
hdfs namenode -format                   # PREMIÈRE FOIS SEULEMENT : crée un HDFS vide
start-dfs.sh                            # NameNode (maître), DataNodes (workers, via SSH), SecondaryNameNode
start-yarn.sh                           # ResourceManager (maître), NodeManagers (workers)
mapred --daemon start historyserver     # historique des jobs terminés
jps                                     # NameNode, SecondaryNameNode, ResourceManager, JobHistoryServer
hdfs dfsadmin -report                   # attendu : Live datanodes (2)
yarn node -list                         # attendu : Total Nodes:2, RUNNING
exit
```

Hôte : vérifier les workers

```
docker exec hadoop-worker1 jps                # DataNode, NodeManager
docker exec hadoop-worker1 python3 --version  # Python 3.10.12 (les tâches Python tournent sur les workers)
docker exec hadoop-master cat /etc/os-release # Ubuntu 22.04.1 LTS
```

Les messages `Permanently added ... to the list of known hosts` correspondent aux connexions SSH du maître vers les workers. `is running as process ... Stop it first` signifie que le service tourne déjà. **Arrêt** dans l'ordre inverse : `mapred --daemon stop historyserver`, `stop-yarn.sh`, `stop-dfs.sh`, puis `docker stop hadoop-master hadoop-worker1 hadoop-worker2`. **Redémarrage** : `docker start hadoop-worker1 hadoop-worker2 hadoop-master`, puis les 3 commandes `start`, sans reformater. Interfaces : http://localhost:9870 et http://localhost:8088.

Étape 3 : contrôler la qualité du CSV

hadoop-master

```
python3 /projet/sources/controle_qualite.py /projet/donnees/ventes_multimagasin_2025.csv \
  | tee /projet/resultats/controle_qualite.txt
```

Le chemin du CSV est passé au script (`sys.argv[1]`) ; `| tee` affiche le résultat et l'enregistre. Attendu : les valeurs de la section 3 (24 000 lignes, 0 manquant, 334 lignes identiques hors identifiant).

Étape 4 : déposer le CSV dans HDFS et prouver son intégrité

hadoop-master

```
ls -l /projet/donnees                                              # 1 985 795 octets
md5sum /projet/donnees/ventes_multimagasin_2025.csv               # empreinte du fichier local
wc -l /projet/donnees/ventes_multimagasin_2025.csv                # 24001
hdfs dfs -mkdir -p /projet_ventes/entree /projet_ventes/entree_blocs_1m /projet_ventes/tests /projet_ventes/
sorties
hdfs dfs -put /projet/donnees/ventes_multimagasin_2025.csv /projet_ventes/entree/
hdfs dfs -D dfs.blocksize=1048576 -put /projet/donnees/ventes_multimagasin_2025.csv /projet_ventes/
entree_blocs_1m/
hdfs dfs -ls -R /projet_ventes
hdfs dfs -cat /projet_ventes/entree/ventes_multimagasin_2025.csv | wc -l      # 24001
hdfs dfs -cat /projet_ventes/entree/ventes_multimagasin_2025.csv | md5sum     # même empreinte
hdfs fsck /projet_ventes/entree/ventes_multimagasin_2025.csv -files -blocks -locations
hdfs fsck /projet_ventes/entree_blocs_1m/ventes_multimagasin_2025.csv -files -blocks -locations
```

Commande	Explication et résultat attendu
`md5sum` , `wc -l`	Empreinte (32 caractères, change si un seul octet change) et nombre de lignes du fichier local : `4ae57ed472eb5c10dd07814deba2c37b` , 24001
`hdfs dfs -mkdir -p`	Crée les dossiers dans HDFS (une entrée dans l'annuaire du NameNode) ; `-p` crée aussi les parents
`hdfs dfs -put`	Copie de Linux vers HDFS : le NameNode choisit 2 DataNodes, le bloc est envoyé au premier qui le retransmet au second (pipeline de réplication)
`-D dfs.blocksize=1048576`	Change la taille de bloc pour cette commande seulement (1 Mo) : le fichier occupe 2 blocs
`hdfs dfs -ls -R`	2e colonne = nombre de répliques (2) ; taille 1 985 795
`hdfs dfs -cat ... | wc -l / md5sum`	Relit la copie HDFS (blocs réassemblés) : mêmes valeurs qu'en local, copie fidèle à l'octet près
`hdfs fsck -files -blocks -locations`	1 bloc (puis 2) ; `Live_repl=2` sur 172.18.0.2 et 172.18.0.3 ; `Status: HEALTHY`

Étape 5 : écrire, tester et lancer les jobs MapReduce

Les scripts sont dans `sources/` (annexe B). On teste toujours en local avant Hadoop : `sort` imite le Shuffle & Sort, et `LC_ALL=C` trie octet par octet comme Hadoop.

hadoop-master : tests locaux

```
head -4 /projet/donnees/ventes_multimagasin_2025.csv | python3 /projet/sources/mapper_detail.py
#   2025-08|M05|Toronto|Bureau|P007|Lot de stylos   801
python3 /projet/sources/mapper_detail.py < /projet/donnees/ventes_multimagasin_2025.csv | wc -l      # 22576
python3 /projet/sources/mapper_detail.py < /projet/donnees/ventes_multimagasin_2025.csv \
  | LC_ALL=C sort | python3 /projet/sources/reducer_somme.py | awk -F'\t' '{s += $2} END {print s}'   #
194661314
```

hadoop-master : job 1 (CSV → ca_detail)

```
mapred streaming \
  -D mapreduce.job.name="ca_detail_r1" -D mapreduce.job.reduces=1 \
  -files /projet/sources/mapper_detail.py,/projet/sources/reducer_somme.py \
  -mapper "python3 mapper_detail.py" -reducer "python3 reducer_somme.py" \
  -input /projet_ventes/entree/ventes_multimagasin_2025.csv \
  -output /projet_ventes/sorties/ca_detail/r1
hdfs dfs -cat /projet_ventes/sorties/ca_detail/r1/part-00000 | wc -l                                  # 1917
hdfs dfs -cat /projet_ventes/sorties/ca_detail/r1/part-00000 | awk -F'\t' '{s += $2} END {print s}'    #
194661314
```

Option	Rôle
`mapred streaming`	Lance un job MapReduce dont le Mapper et le Reducer sont des programmes quelconques (ici Python)
`-D mapreduce.job.name`	Nom du job affiché dans YARN (http://localhost:8088)
`-D mapreduce.job.reduces`	Nombre de Reducers, donc de fichiers `part-*`
`-files`	Chemins **Linux** des scripts, copiés sur chaque worker
`-mapper` , `-reducer`	Commandes exécutées dans les conteneurs YARN
`-input` , `-output`	Chemins **HDFS** ; le dossier de sortie ne doit pas exister (sinon : supprimer avec `hdfs dfs -rm -r`)

hadoop-master : jobs 2 à 5 (chaînage), contrôle et récupération

```
for niveau in mois magasin categorie produit
do
  mapred streaming \
    -D mapreduce.job.name="ca_${niveau}_r1" -D mapreduce.job.reduces=1 \
    -files /projet/sources/mapper_regroupe.py,/projet/sources/reducer_somme.py \
    -mapper "python3 mapper_regroupe.py ${niveau}" -reducer "python3 reducer_somme.py" \
    -input /projet_ventes/sorties/ca_detail/r1 \
    -output /projet_ventes/sorties/ca_${niveau}/r1
done
for niveau in mois magasin categorie produit; do
  hdfs dfs -cat /projet_ventes/sorties/ca_${niveau}/r1/part-00000 | wc -l                 # 12, 8, 4, 20
done
# contrôle : CA mensuel calculé directement depuis le CSV avec mapper_mois.py
mapred streaming -D mapreduce.job.name="ca_mensuel_r1" -D mapreduce.job.reduces=1 \
  -files /projet/sources/mapper_mois.py,/projet/sources/reducer_somme.py \
  -mapper "python3 mapper_mois.py" -reducer "python3 reducer_somme.py" \
  -input /projet_ventes/entree/ventes_multimagasin_2025.csv \
  -output /projet_ventes/sorties/controle_ca_mensuel_direct/r1
hdfs dfs -cat /projet_ventes/sorties/ca_mois/r1/part-00000 | md5sum
hdfs dfs -cat /projet_ventes/sorties/controle_ca_mensuel_direct/r1/part-00000 | md5sum   # identiques
mkdir -p /projet/resultats
for job in detail mois magasin categorie produit; do
  hdfs dfs -get /projet_ventes/sorties/ca_${job}/r1 /projet/resultats/ca_${job}
done
```

`for ... do ... done` répète le bloc pour chaque niveau ; `${niveau}` est remplacé par sa valeur, et le script le lit avec `sys.argv[1]`. `-input` désigne le dossier du job 1 : Hadoop lit ses `part-*` et ignore `_SUCCESS`. `hdfs dfs -get`

copie de HDFS vers le dossier `resultats/` du projet, lu par le tableau de bord. Le message `Unable to write to output stream` après un `| head` est sans gravité : `head` ferme le pipe.

Étape 6 : expérimenter (Reducers, Mappers, YARN)

hadoop-master

```
# 2 puis 4 Reducers
for r in 2 4; do
  mapred streaming -D mapreduce.job.name="ca_mois_r${r}" -D mapreduce.job.reduces=${r} \
    -files /projet/sources/mapper_regroupe.py,/projet/sources/reducer_somme.py \
    -mapper "python3 mapper_regroupe.py mois" -reducer "python3 reducer_somme.py" \
    -input /projet_ventes/sorties/ca_detail/r1 -output /projet_ventes/sorties/ca_mois/r${r}
done
for f in $(hdfs dfs -ls -C /projet_ventes/sorties/ca_mois/r4/part-*); do echo "== $f"; hdfs dfs -cat $f; done
hdfs dfs -cat /projet_ventes/sorties/ca_mois/r4/part-* | awk -F'\t' '{s += $2} END {print s}'   # 194661314
# 1 puis 4 Mappers demandés (fichier à 1 bloc), puis 1 Mapper demandé sur le fichier à 2 blocs
for m in 1 4; do
  mapred streaming -D mapreduce.job.name="exp_maps${m}" -D mapreduce.job.maps=${m} -D mapreduce.job.reduces=1
\
    -files /projet/sources/mapper_detail.py,/projet/sources/reducer_somme.py \
    -mapper "python3 mapper_detail.py" -reducer "python3 reducer_somme.py" \
    -input /projet_ventes/entree/ventes_multimagasin_2025.csv -output /projet_ventes/sorties/exp_mappers/
maps${m}
done
mapred streaming -D mapreduce.job.name="exp_blocs1m_maps1" -D mapreduce.job.maps=1 -D mapreduce.job.reduces=1
\
  -files /projet/sources/mapper_detail.py,/projet/sources/reducer_somme.py \
  -mapper "python3 mapper_detail.py" -reducer "python3 reducer_somme.py" \
  -input /projet_ventes/entree_blocs_1m/ventes_multimagasin_2025.csv \
  -output /projet_ventes/sorties/exp_mappers/blocs1m_maps1
# ressources YARN
yarn node -list -showDetails                          # Configured Resources :
yarn application -status application_<id>            # Aggregate Resource Allocation, AM Host
```

Dans chaque journal, relever : `number of splits`, `Launched map tasks`, `Launched reduce tasks`, `Shuffled Maps`, `Bytes Read`, `Physical memory snapshot`, `megabyte-milliseconds` et `vcore-milliseconds`, et les heures de début et de fin. `$( ... )` remplace une commande par son résultat ; `-ls -C` n'affiche que les chemins.

Étape 7 : lancer le tableau de bord en local

Hôte

```
cd application_python
python -m pip install -r requirements.txt     # streamlit 1.66.1, plotly 7.1.0, pandas 3.1.0
python -m streamlit run app.py                # http://localhost:8501 ; arrêt : Ctrl + C
```

L'application cherche les résultats dans `../resultats` (ou dans le dossier de la variable `DOSSIER_RESULTATS`). Attendu sans filtre : CA 1 946 613 $, évolution +86 %, meilleur mois déc., meilleur magasin M05 Toronto ; avec Ottawa + Montréal : 926 379 $.

Étape 8 : rejouer le cas de test

hadoop-master

```
python3 -c "print(round(6.50 * 0.95, 2))"       # 6.17 : le piège des flottants
python3 /projet/sources/mapper_detail.py < /projet/tests/echantillon_test.csv | LC_ALL=C sort \
  | python3 /projet/sources/reducer_somme.py | diff - /projet/tests/attendu_ca_detail.txt && echo "TEST LOCAL
DETAIL : REUSSI"
hdfs dfs -put /projet/tests/echantillon_test.csv /projet_ventes/tests/
mapred streaming -D mapreduce.job.name="test_ca_detail" -D mapreduce.job.reduces=1 \
  -files /projet/sources/mapper_detail.py,/projet/sources/reducer_somme.py \
  -mapper "python3 mapper_detail.py" -reducer "python3 reducer_somme.py" \
  -input /projet_ventes/tests/echantillon_test.csv -output /projet_ventes/sorties/tests/ca_detail
hdfs dfs -cat /projet_ventes/sorties/tests/ca_detail/part-* | diff - /projet/tests/attendu_ca_detail.txt &&
echo "TEST HADOOP DETAIL : REUSSI"
mapred streaming -D mapreduce.job.name="test_ca_mois" -D mapreduce.job.reduces=1 \
  -files /projet/sources/mapper_regroupe.py,/projet/sources/reducer_somme.py \
  -mapper "python3 mapper_regroupe.py mois" -reducer "python3 reducer_somme.py" \
  -input /projet_ventes/sorties/tests/ca_detail -output /projet_ventes/sorties/tests/ca_mois
hdfs dfs -cat /projet_ventes/sorties/tests/ca_mois/part-* | diff - /projet/tests/attendu_ca_mois.txt && echo
"TEST HADOOP CHAINAGE : REUSSI"
```

`diff - fichier` compare la sortie reçue par le pipe avec le résultat attendu et n'affiche rien s'ils sont identiques ; `&&` n'affiche le message que si `diff` a réussi.

Étape 9 : construire, tester et publier l'image Docker

Hôte (dans application_python)

```
docker build -t yacinelezoul/dashboard-ventes:1.0 .
docker run -d --name dashboard-ventes -p 8501:8501 -v "$(pwd)/../resultats:/app/resultats:ro" yacinelezoul/
dashboard-ventes:1.0
docker logs dashboard-ventes                 # en cas de problème
docker login                                 # connexion par le navigateur (code à confirmer)
docker push yacinelezoul/dashboard-ventes:1.0 # note le digest sha256:...
docker rm -f dashboard-ventes
docker rmi yacinelezoul/dashboard-ventes:1.0
docker pull yacinelezoul/dashboard-ventes:1.0 # Downloaded newer image, même digest
docker run -d --name dashboard-ventes -p 8501:8501 -v "$(pwd)/../resultats:/app/resultats:ro" yacinelezoul/
dashboard-ventes:1.0
docker ps
```

`docker build -t` construit l'image selon le Dockerfile, à partir du dossier courant (`.`). `-v ...:/app/resultats:ro` donne les sorties MapReduce au conteneur, en lecture seule. Le port 8501 doit être libre (arrêter un `streamlit run` local). Ne jamais écrire de mot de passe ni de jeton dans un fichier : le dépôt est public.

Étape 10 : versionner et préparer les livrables

Hôte (racine du projet)

```
git add .
git commit -m "description courte de la modification"
git push
```

`.gitignore` exclut `__pycache__` et les environnements virtuels ; `.gitattributes` impose des fins de ligne Unix aux scripts (`*.py text eol=lf`) et interdit toute conversion du CSV (`*.csv -text`), pour que son md5 reste identique

chez chaque membre. Livrables à la racine : `rapport.pdf` (rapport d'évaluation), `rapport_detaille.pdf`, `presentation.pptx`, `README.md`, `preuve_docker.txt`.

ANNEXE B

Code source commenté

En bref Code complet des scripts MapReduce, du contrôle qualité et de l'image Docker, tel que versionné dans le dépôt GitHub. Les commentaires sont dans le code ; chaque bloc est précédé de son rôle dans la chaîne. Le fichier `app.py` (282 lignes) est décrit par sa structure en 7 parties ; il est complet dans le dépôt.

B.1 Scripts MapReduce et contrôle (dossier sources/)

mapper_detail.py : Mapper du job 1

Lit le CSV ligne par ligne sur l'entrée standard, ignore l'en-tête, les lignes mal formées et les ventes ANNULEE, calcule le montant net en cents avec `Decimal` et l'arrondi vers le haut, puis émet la clé composite `mois|magasin|ville|categorie|id_produit|nom_produit` et le montant, séparés par une tabulation.

sources/mapper_detail.py

```
import sys
from decimal import Decimal, ROUND_HALF_UP

# même début que mapper_mois.py : lecture, filtres, calcul du montant
for ligne in sys.stdin:
    ligne = ligne.strip()
    if ligne == "" or ligne.startswith("id_vente"):
        continue
    champs = ligne.split(",")
    if len(champs) != 12:
        continue
    if champs[10] != "VALIDEE":
        continue
    quantite = int(champs[7])
    prix = Decimal(champs[8])
    remise = Decimal(champs[9])
    montant = quantite * prix * (1 - remise / 100)
    cents = int((montant * 100).quantize(Decimal("1"), rounding=ROUND_HALF_UP))

    # on récupère toutes les colonnes utiles à la clé
    mois = champs[1][:7]
    magasin = champs[2]
    ville = champs[3]
    id_produit = champs[4]
    nom_produit = champs[5]
    categorie = champs[6]

    # SEULE DIFFÉRENCE avec mapper_mois.py : la clé contient tout, séparé par |
    cle = mois + "|" + magasin + "|" + ville + "|" + categorie + "|" + id_produit + "|" + nom_produit
    print(f"{cle}\t{cents}")
```

mapper_regroupe.py : Mapper des jobs 2 à 5

Lit les lignes de `ca_detail`, découpe la clé composite et ne garde que les morceaux du niveau demandé en argument (`mois`, `magasin`, `categorie` ou `produit`). La valeur en cents est recopiée telle quelle.

sources/mapper_regroupe.py

```
import sys

# le niveau demandé est écrit après le nom du script :
# python3 mapper_regroupe.py categorie  ->  niveau = "categorie"
niveau = sys.argv[1]

# chaque ligne reçue vient de ca_detail : "2025-01|M01|Ottawa|Bureau|P006|Cahier A5<TAB>12709"
for ligne in sys.stdin:
    ligne = ligne.strip()
    if ligne == "":
        continue
    cle_detail, cents = ligne.split("\t")     # sépare la clé détaillée et le montant
    parties = cle_detail.split("|")           # découpe la clé en 6 morceaux
    mois = parties[0]
    magasin = parties[1]
    ville = parties[2]
    categorie = parties[3]
    id_produit = parties[4]
    nom_produit = parties[5]

    # on ne garde que la partie de la clé qui correspond au niveau demandé
    if niveau == "mois":
        cle = mois
    elif niveau == "magasin":
        cle = magasin + "|" + ville
    elif niveau == "categorie":
        cle = categorie
    elif niveau == "produit":
        cle = id_produit + "|" + nom_produit + "|" + categorie
    else:
        sys.exit("niveau inconnu : " + niveau)

    print(f"{cle}\t{cents}")
```

reducer_somme.py : Reducer unique des 6 jobs

Reçoit les lignes triées par clé (Shuffle & Sort). Tant que la clé ne change pas, il additionne ; au changement de clé et à la fin, il écrit le total. Il ne dépend pas du sens de la clé, d'où sa réutilisation pour tous les jobs.

sources/reducer_somme.py

```
import sys

cle_courante = None     # la clé qu'on est en train d'additionner (ex. "2025-03")
total = 0               # le total en cours pour cette clé

# Hadoop envoie les paires TRIÉES par clé : tous les "2025-03" arrivent à la suite
for ligne in sys.stdin:
    ligne = ligne.strip()
    if ligne == "":
        continue

    cle, valeur = ligne.split("\t")    # sépare la clé et la valeur (tabulation)
    valeur = int(valeur)               # "801" devient le nombre 801

    if cle == cle_courante:
        # même clé que la ligne précédente : on ajoute au total
        total = total + valeur
    else:
        # nouvelle clé : on écrit le total de la clé précédente...
        if cle_courante is not None:
            print(f"{cle_courante}\t{total}")
        # ...puis on recommence à compter pour la nouvelle clé
        cle_courante = cle
        total = valeur

# à la fin, on écrit le total de la dernière clé
if cle_courante is not None:
    print(f"{cle_courante}\t{total}")
```

mapper_mois.py : Mapper du job de contrôle

Calcule le CA mensuel directement depuis le CSV, avec les mêmes règles que `mapper_detail.py`. Son résultat doit être identique (même md5) à celui du chaînage `ca_detail` vers `ca_mois`.

sources/mapper_mois.py

```
import sys
from decimal import Decimal, ROUND_HALF_UP

# Hadoop envoie les lignes du CSV une par une au script
for ligne in sys.stdin:
    ligne = ligne.strip()                    # enlève le retour à la ligne

    # on saute l'en-tête et les lignes vides
    if ligne == "" or ligne.startswith("id_vente"):
        continue

    champs = ligne.split(",")                # découpe la ligne en 12 colonnes
    if len(champs) != 12:                    # ligne abîmée : on l'ignore
        continue
    if champs[10] != "VALIDEE":              # colonne statut : on ignore les annulées
        continue

    quantite = int(champs[7])                # colonne quantite
    prix = Decimal(champs[8])                # colonne prix_unitaire_cad (nombre exact)
    remise = Decimal(champs[9])              # colonne remise_pct

    montant = quantite * prix * (1 - remise / 100)
    # arrondi au cent, 0,5 vers le haut, puis conversion en cents entiers
    cents = int((montant * 100).quantize(Decimal("1"), rounding=ROUND_HALF_UP))

    mois = champs[1][:7]                     # "2025-08-08" devient "2025-08"

    # la paire clé-valeur : mois, tabulation, montant en cents
    print(f"{mois}\t{cents}")
```

controle_qualite.py : contrôle de l'étape 3

Vérifie le nombre de colonnes, les valeurs manquantes, les doublons d'identifiant et de contenu, les valeurs des colonnes catégorielles, les bornes des valeurs numériques et le nombre de ventes au demi-cent.

sources/controle_qualite.py

```
import sys
import csv
from datetime import date
from decimal import Decimal, InvalidOperation

# le chemin du CSV est donné après le nom du script
chemin = sys.argv[1]

COLONNES = ["id_vente", "date_vente", "id_magasin", "ville", "id_produit", "nom_produit",
            "categorie", "quantite", "prix_unitaire_cad", "remise_pct", "statut", "mode_paiement"]
REMISES_PERMISES = {"0", "5", "10", "15", "20"}
STATUTS_PERMIS = {"VALIDEE", "ANNULEE"}

def titre(texte):
    print()
    print("== " + texte)

# ---------- 1. Lecture : le fichier doit être en UTF-8 ----------
with open(chemin, encoding="utf-8", newline="") as f:
    lecteur = csv.reader(f)
    entete = next(lecteur)
    lignes = list(lecteur)

titre("1. Structure")
print("En-tête conforme au dictionnaire :", entete == COLONNES)
print("Lignes de données :", len(lignes))
mal_formees = [l for l in lignes if len(l) != 12]
print("Lignes qui n'ont pas 12 colonnes :", len(mal_formees))
lignes = [l for l in lignes if len(l) == 12]

# ---------- 2. Valeurs manquantes ----------
titre("2. Valeurs manquantes par colonne")
for i, nom in enumerate(COLONNES):
    vides = sum(1 for l in lignes if l[i].strip() == "")
    print(f"{nom:<18} {vides}")

# ---------- 3. Doublons ----------
titre("3. Doublons")
ids = [l[0] for l in lignes]
print("id_vente en double :", len(ids) - len(set(ids)))
print("Lignes entières en double :", len(lignes) - len(set(tuple(l) for l in lignes)))
sans_id = [tuple(l[1:]) for l in lignes]
print("Lignes identiques hors id_vente :", len(sans_id) - len(set(sans_id)))

# ---------- 4. Dates ----------
titre("4. Dates")
dates, invalides = [], 0
for l in lignes:
    try:
        dates.append(date.fromisoformat(l[1]))
    except ValueError:
        invalides += 1
print("Dates invalides :", invalides)
print("Période :", min(dates), "au", max(dates))

# ---------- 5. Cohérence des référentiels ----------
titre("5. Cohérence magasins et produits")
villes_par_magasin, fiche_produit = {}, {}
for l in lignes:
    villes_par_magasin.setdefault(l[2], set()).add(l[3])
    fiche_produit.setdefault(l[4], set()).add((l[5], l[6], l[8]))
print("Magasins :", len(villes_par_magasin), "| villes :", len({l[3] for l in lignes}))
print("Magasins avec plus d'une ville :", sum(1 for v in villes_par_magasin.values() if len(v) > 1))
print("Produits :", len(fiche_produit), "| catégories :", sorted({l[6] for l in lignes}))
print("Produits avec plusieurs noms, catégories ou prix :", sum(1 for v in fiche_produit.values() if len(v) >
```

```
1))

# ---------- 6. Valeurs numériques et codes ----------
titre("6. Valeurs numériques et codes")
quantites, prix = [], []
qte_invalides = prix_invalides = 0
for l in lignes:
    try:
        q = int(l[7])
        if q < 1:
            qte_invalides += 1
        quantites.append(q)
    except ValueError:
        qte_invalides += 1
    try:
        p = Decimal(l[8])
        if p <= 0:
            prix_invalides += 1
        prix.append(p)
    except InvalidOperation:
        prix_invalides += 1
print("quantite : de", min(quantites), "à", max(quantites), "| invalides :", qte_invalides)
print("prix_unitaire_cad : de", min(prix), "à", max(prix), "| invalides :", prix_invalides)
print("remise_pct hors {0, 5, 10, 15, 20} :", sum(1 for l in lignes if l[9] not in REMISES_PERMISES))
print("statut hors {VALIDEE, ANNULEE} :", sum(1 for l in lignes if l[10] not in STATUTS_PERMIS))

# ---------- 7. Répartitions ----------
titre("7. Répartitions")
for i, nom in [(10, "statut"), (11, "mode_paiement"), (2, "id_magasin")]:
    compte = {}
    for l in lignes:
        compte[l[i]] = compte.get(l[i], 0) + 1
    print(nom, ":", dict(sorted(compte.items())))
```

B.2 Lecture des résultats et image Docker (dossier application_python/)

lecture_resultats.py : lecture des sorties MapReduce

Trouve le dossier des résultats (variable d'environnement `DOSSIER_RESULTATS` dans l'image, `../resultats` en local), lit tous les fichiers `part-*` d'un job, découpe la clé selon `COLONNES_CLE` et convertit les cents en dollars. Le tableau de bord ne lit jamais le CSV.

application_python/lecture_resultats.py

```
import os
from pathlib import Path

import pandas as pd

# Dossier des résultats MapReduce :
# - sur Windows : Projet\resultats (le dossier voisin de application_python)
# - dans Docker : la variable d'environnement DOSSIER_RESULTATS (ex. /app/resultats)
DOSSIER_PAR_DEFAUT = Path(__file__).resolve().parent.parent / "resultats"
DOSSIER = Path(os.environ.get("DOSSIER_RESULTATS", DOSSIER_PAR_DEFAUT))

# Pour chaque job : les noms des morceaux de la clé, séparés par |
COLONNES_CLE = {
    "ca_mois": ["mois"],
    "ca_magasin": ["magasin", "ville"],
    "ca_categorie": ["categorie"],
    "ca_produit": ["id_produit", "nom_produit", "categorie"],
    "ca_detail": ["mois", "magasin", "ville", "categorie", "id_produit", "nom_produit"],
}


def fichiers_part(job):
    """Liste les fichiers part-* d'un job (un par Reducer)."""
    return sorted((DOSSIER / job).glob("part-*"))


def lire_job(job):
    """Lit tous les part-* d'un job et renvoie un tableau : colonnes de la clé + ca (en $)."""
    lignes = []
    for fichier in fichiers_part(job):
        with open(fichier, encoding="utf-8") as f:
            for ligne in f:
                ligne = ligne.strip()
                if ligne == "":
                    continue
                cle, cents = ligne.split("\t")
                lignes.append(cle.split("|") + [int(cents)])
    tableau = pd.DataFrame(lignes, columns=COLONNES_CLE[job] + ["cents"])
    tableau["ca"] = tableau["cents"] / 100     # cents -> dollars, seulement pour l'affichage
    return tableau


def provenance():
    """Pour chaque job : les fichiers lus et leur nombre de lignes."""
    infos = []
    for job in COLONNES_CLE:
        for fichier in fichiers_part(job):
            with open(fichier, encoding="utf-8") as f:
                nb = sum(1 for _ in f)
            infos.append({"job": job, "fichier": fichier.name, "lignes": nb})
    return pd.DataFrame(infos)
```

Dockerfile

Image de base Python légère ; les dépendances sont installées avant la copie du code pour profiter du cache Docker (deuxième build en 1,4 s) ; les résultats ne sont pas copiés dans l'image, ils sont montés au lancement ou lus dans `/app/resultats`.

application_python/Dockerfile

```
# image de départ : Linux minimal avec Python 3.14 (même version que sous Windows)
FROM python:3.14-slim

# dossier de travail dans l'image
WORKDIR /app

# 1) les dépendances d'abord : cette couche est réutilisée tant que requirements.txt ne change pas
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 2) le code de l'application et son thème
COPY app.py lecture_resultats.py ./
COPY .streamlit/ ./.streamlit/

# l'application lira les résultats MapReduce dans ce dossier (fourni par un volume au lancement)
ENV DOSSIER_RESULTATS=/app/resultats

# port de Streamlit
EXPOSE 8501

# commande lancée au démarrage du conteneur
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501", "--
server.headless=true"]
```

requirements.txt

Versions figées : le même build donne la même image chez chaque membre.

application_python/requirements.txt

```
streamlit==1.66.1
plotly==7.1.0
pandas==3.1.0
```

.dockerignore

Fichiers exclus du contexte de build (cache Python, fichiers Docker eux-mêmes).

application_python/.dockerignore

```
__pycache__/
*.pyc
Dockerfile
.dockerignore
```

.streamlit/config.toml

Thème clair et barre d'outils réduite.

application_python/.streamlit/config.toml

```
[theme]
base = "light"
primaryColor = "#4F46E5"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F3F4F8"
textColor = "#1F2937"

[client]
toolbarMode = "minimal"
```

app.py : structure en 7 parties

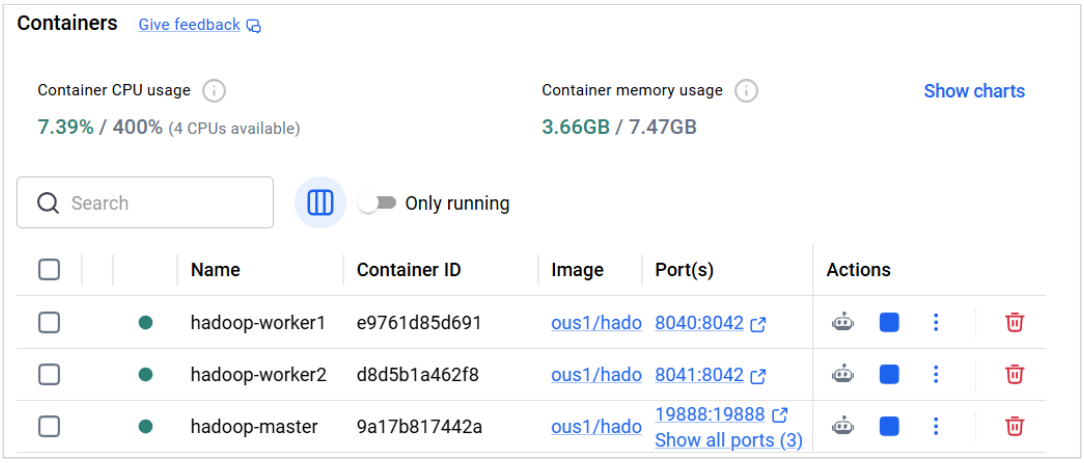
Partie	Lignes et contenu	Rôle
1	7 à 42 : réglages, `dollars()`, `texte_md()`, `mise_en_forme()`	Noms des mois, couleur fixe par ville et par catégorie, format « 1 946 613,14 $ », échappement du `$` pour Markdown, style commun des graphiques Plotly
2	43 à 52 : lecture	Charge `ca_detail` et les 4 jobs agrégés par `lecture_resultats.lire_job`, avec cache Streamlit
3	53 à 73 : filtres	Barre latérale : ville, magasin (limité aux villes choisies), catégorie, période ; un filtre vide signifie « toutes les valeurs »
4	74 à 106 : indicateurs	4 cartes : CA de la sélection et part du total, évolution janvier à décembre, meilleur mois, meilleur magasin ; création des 3 onglets
5	107 à 188 : onglet Analyses	Les 4 analyses obligatoires (CA mensuel, par magasin, par catégorie, top 5 produits), recalculées sur `ca_detail` filtré
6	189 à 252 : tableau croisé	Fonction récursive `lignes_tcd()` : hiérarchie ville, magasin, catégorie en accordéons, mois en colonnes, totaux
7	253 à 282 : provenance	Dossier lu, fichiers `part-*`, nombre de lignes, total de contrôle de chaque job comparé à `ca_detail`

ANNEXE C

Preuves d'exécution

En bref Captures des interfaces (Docker, HDFS, YARN, tableau de bord) et valeurs relevées dans les journaux. Les fichiers originaux sont dans `captures/` et `scripts_execution/journaux/` (01 à 11) du dépôt ; la preuve de publication de l'image est dans `preuve_docker.txt`.

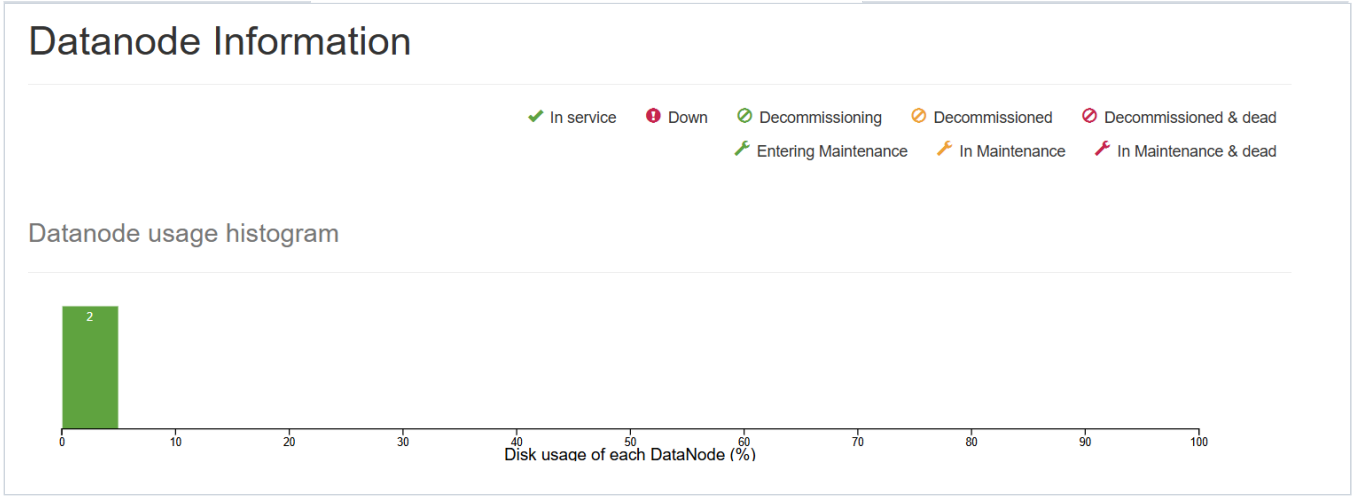
C.1 Cluster et HDFS



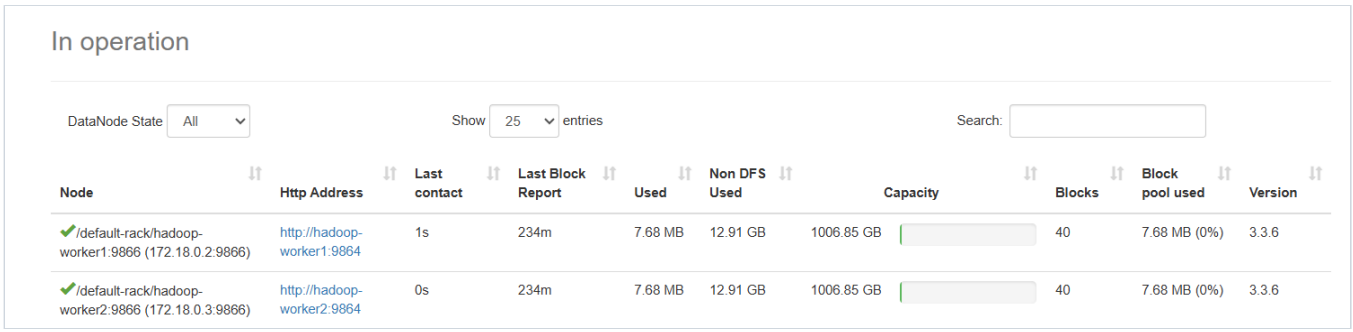
Capture C1. *Docker Desktop : hadoop-master, hadoop-worker1 et hadoop-worker2 en cours d'exécution, ports publiés (9870, 8088, 19888, 8040, 8041)*



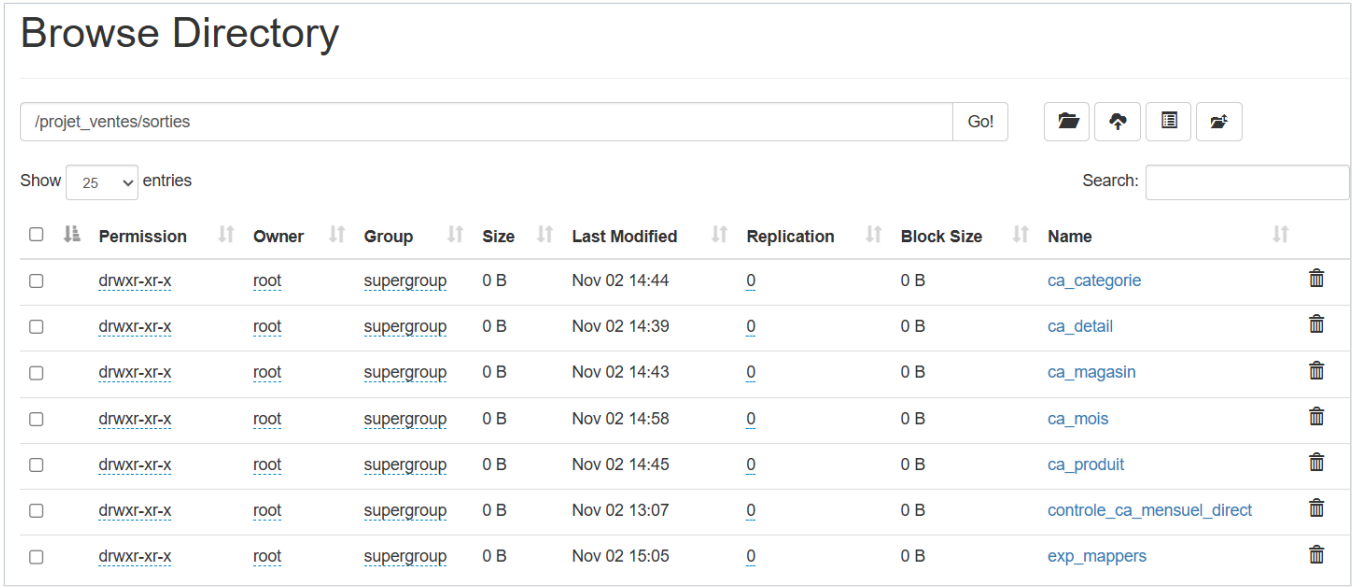
Capture C2. *Interface du NameNode (http://localhost:9870) : Hadoop 3.3.6, cluster actif*



Capture C3. *Onglet Datanodes : utilisation de l'espace par les 2 DataNodes*

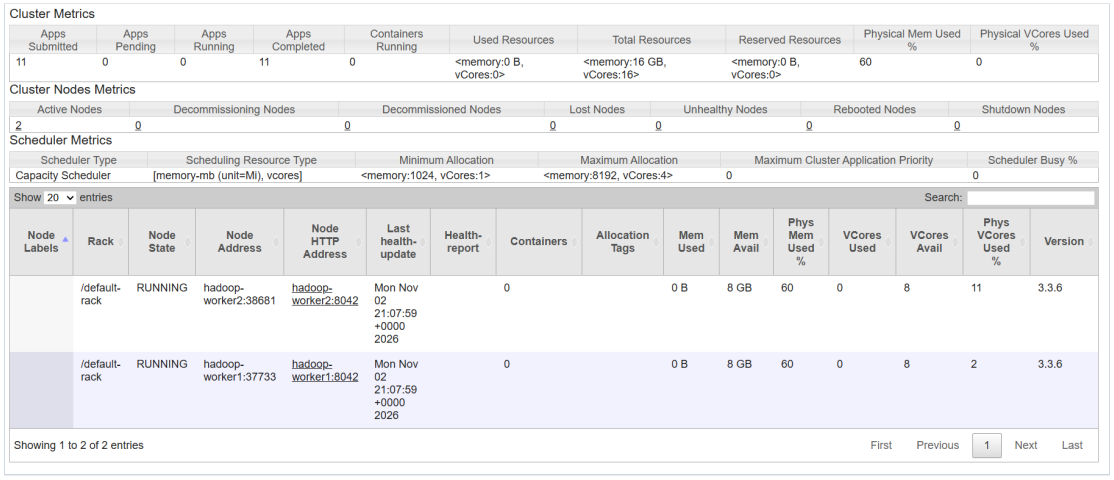


Capture C4. *Les 2 DataNodes en service (In Service), hadoop-worker1 et hadoop-worker2*



Capture C5. *Explorateur HDFS : dossiers de sortie dans* `/projet_ventes/sorties` *(ca_detail, ca_mois, ca_magasin, ca_categorie, ca_produit, contrôle, expériences, tests)*

C.2 YARN



Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources	Reserved Resources	Physical Mem Used %	Physical VCores Used %
11	0	0	11	0	<memory:0 B, vCores:0>	<memory:16 GB, vCores:16>	<memory:0 B, vCores:0>	60	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes	Shutdown Nodes
2	0	0	0	0	0	0

Scheduler Metrics

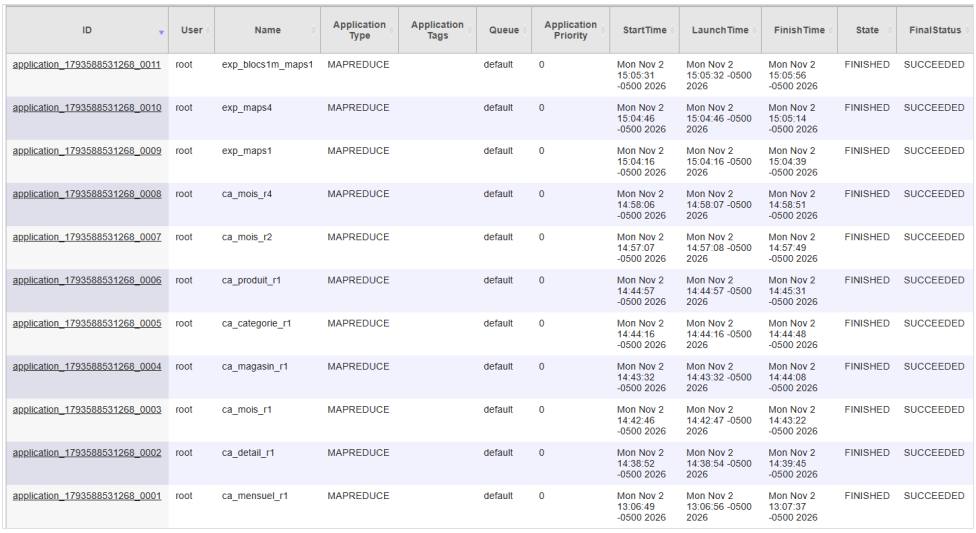
Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluster Application Priority	Scheduler Busy %
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>	0	0

Show 20 entries — Search:

Node Labels	Rack	Node State	Node Address	Node HTTP Address	Last health-update	Health-report	Containers	Allocation Tags	Mem Used	Mem Avail	Phys Mem Used %	VCores Used	VCores Avail	Phys VCores Used %	Version
	/default-rack	RUNNING	hadoop-worker2:38681	hadoop-worker2:8042	Mon Nov 02 21:07:59 +0000 2026		0		0 B	8 GB	60	0	8	11	3.3.6
	/default-rack	RUNNING	hadoop-worker1:37733	hadoop-worker1:8042	Mon Nov 02 21:07:59 +0000 2026		0		0 B	8 GB	60	0	8	2	3.3.6

Showing 1 to 2 of 2 entries — First Previous 1 Next Last

Capture C6. *ResourceManager (http://localhost:8088) : 2 NodeManagers actifs, 8 192 Mo et 8 vCores chacun*



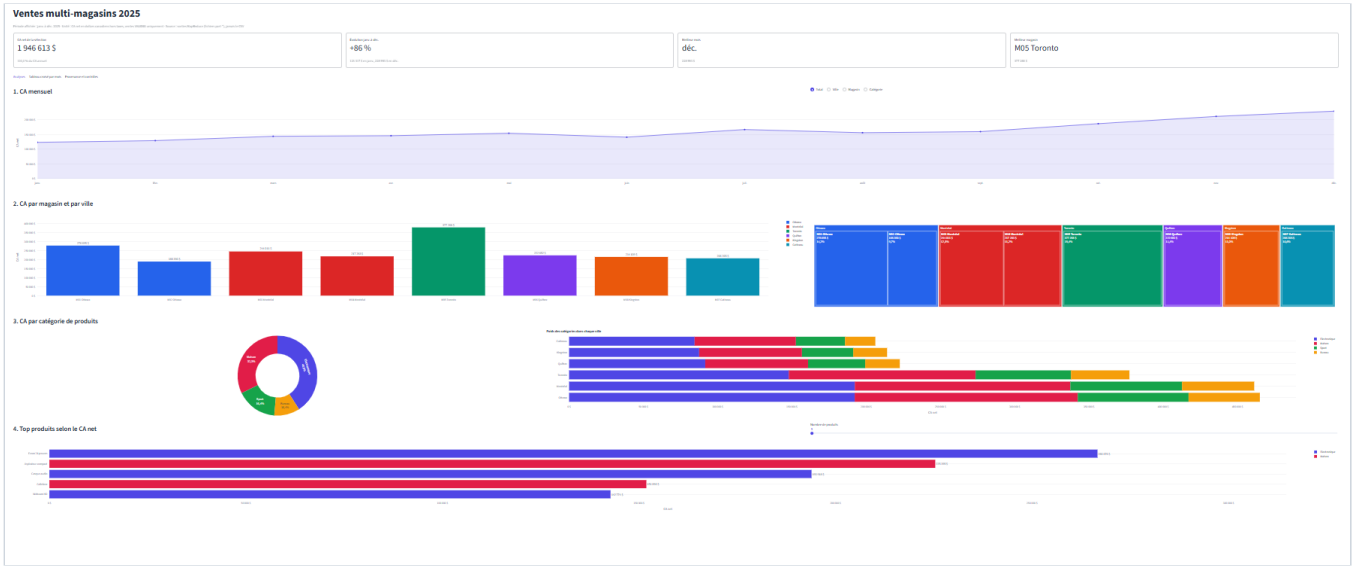
ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus
application_1793588531268_0011	root	exp_blocs1m_maps1	MAPREDUCE		default	0	Mon Nov 2 15:05:31 -0500 2026	Mon Nov 2 15:05:32 -0500 2026	Mon Nov 2 15:05:56 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0010	root	exp_maps4	MAPREDUCE		default	0	Mon Nov 2 15:04:46 -0500 2026	Mon Nov 2 15:04:46 -0500 2026	Mon Nov 2 15:05:14 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0009	root	exp_maps1	MAPREDUCE		default	0	Mon Nov 2 15:04:16 -0500 2026	Mon Nov 2 15:04:16 -0500 2026	Mon Nov 2 15:04:39 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0008	root	ca_mois_r4	MAPREDUCE		default	0	Mon Nov 2 14:58:06 -0500 2026	Mon Nov 2 14:58:07 -0500 2026	Mon Nov 2 14:58:51 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0007	root	ca_mois_r2	MAPREDUCE		default	0	Mon Nov 2 14:57:07 -0500 2026	Mon Nov 2 14:57:08 -0500 2026	Mon Nov 2 14:57:49 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0006	root	ca_produit_r1	MAPREDUCE		default	0	Mon Nov 2 14:44:57 -0500 2026	Mon Nov 2 14:44:57 -0500 2026	Mon Nov 2 14:45:31 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0005	root	ca_categorie_r1	MAPREDUCE		default	0	Mon Nov 2 14:44:16 -0500 2026	Mon Nov 2 14:44:16 -0500 2026	Mon Nov 2 14:44:48 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0004	root	ca_magasin_r1	MAPREDUCE		default	0	Mon Nov 2 14:43:32 -0500 2026	Mon Nov 2 14:43:32 -0500 2026	Mon Nov 2 14:44:08 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0003	root	ca_mois_r1	MAPREDUCE		default	0	Mon Nov 2 14:42:46 -0500 2026	Mon Nov 2 14:42:47 -0500 2026	Mon Nov 2 14:43:22 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0002	root	ca_detail_r1	MAPREDUCE		default	0	Mon Nov 2 14:38:52 -0500 2026	Mon Nov 2 14:38:54 -0500 2026	Mon Nov 2 14:39:45 -0500 2026	FINISHED	SUCCEEDED
application_1793588531268_0001	root	ca_mensuel_r1	MAPREDUCE		default	0	Mon Nov 2 13:06:49 -0500 2026	Mon Nov 2 13:06:56 -0500 2026	Mon Nov 2 13:07:37 -0500 2026	FINISHED	SUCCEEDED

Capture C7. *Liste des applications : les 13 jobs du projet à l'état FINISHED / SUCCEEDED*

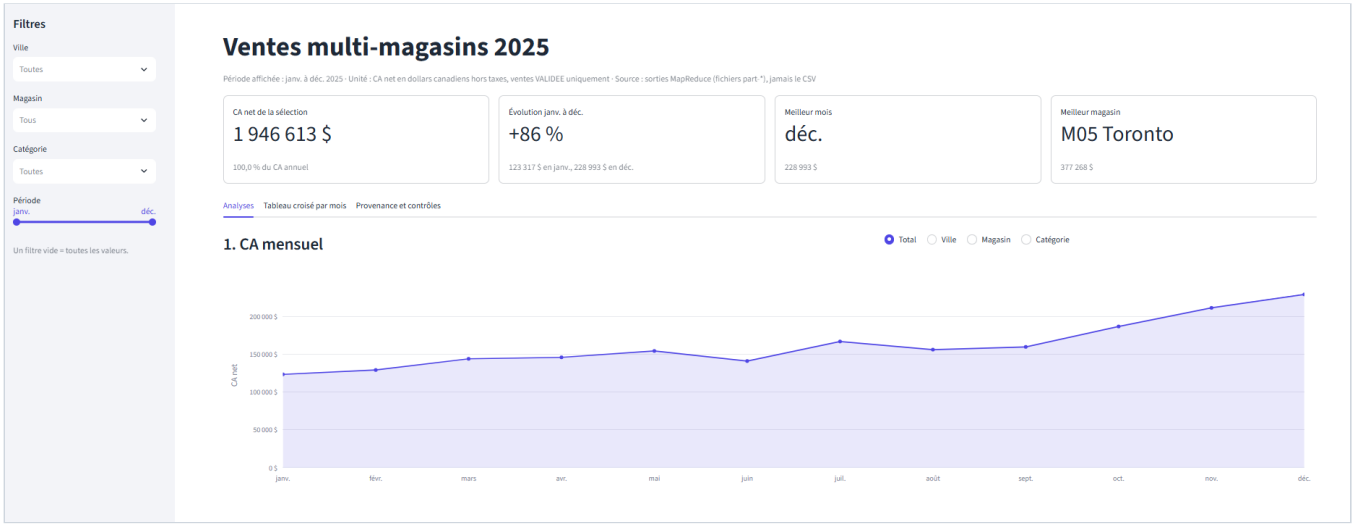
Job (nom YARN)	Rôle	Résultat
ca_detail_r1	Job 1, CSV vers clé détaillée	1 917 lignes
ca_mois_r1, ca_magasin_r1, ca_categorie_r1, ca_produit_r1	Jobs 2 à 5, chaînés sur ca_detail	12, 8, 4, 20 lignes
ca_mensuel_r1 (controle_ca_mensuel_direct)	Contrôle direct depuis le CSV	**md5 identique**
ca_mois_r2, ca_mois_r4	Expérience Reducers	27 s, 34 s
exp_maps1, exp_maps4, exp_blocs1m_maps1	Expérience Mappers	1, 4, 2 Mappers
test_ca_detail, test_ca_mois	Cas de test sur Hadoop	**REUSSI**

Identifiants : `application_1793588531268_0001` à `_0013`, tous SUCCEEDED.

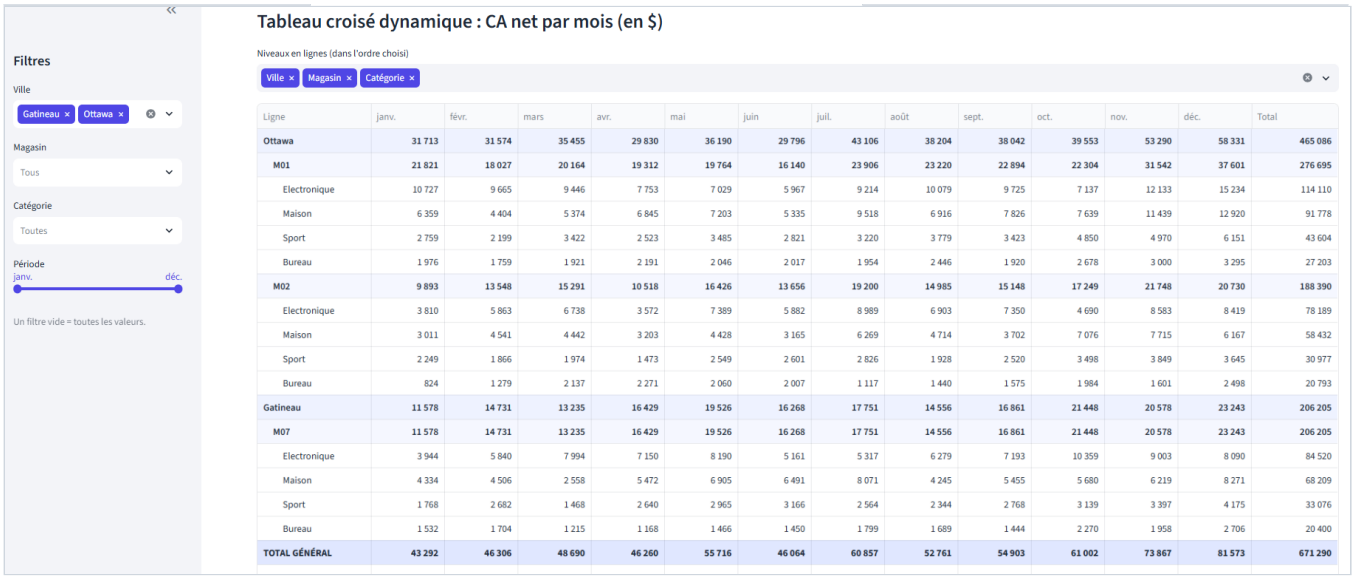
C.3 Tableau de bord



Capture C8. *Onglet Analyses, vue complète : filtres, 4 indicateurs et graphiques*

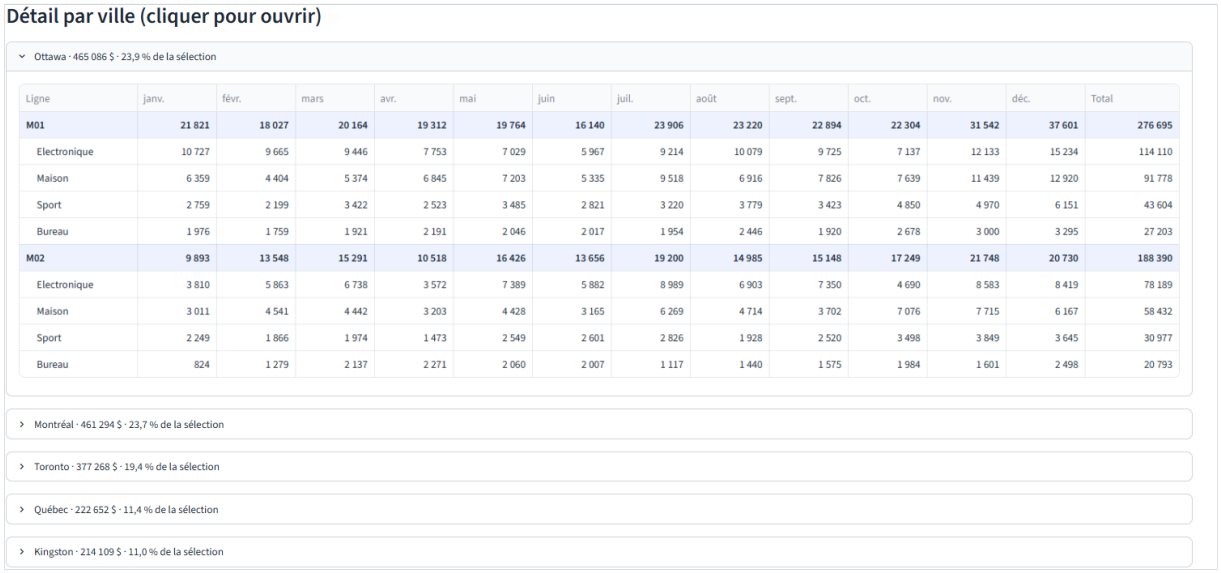


Capture C9. *Sans filtre : CA 1 946 613 $, évolution +86 %, meilleur mois décembre, meilleur magasin M05 Toronto*



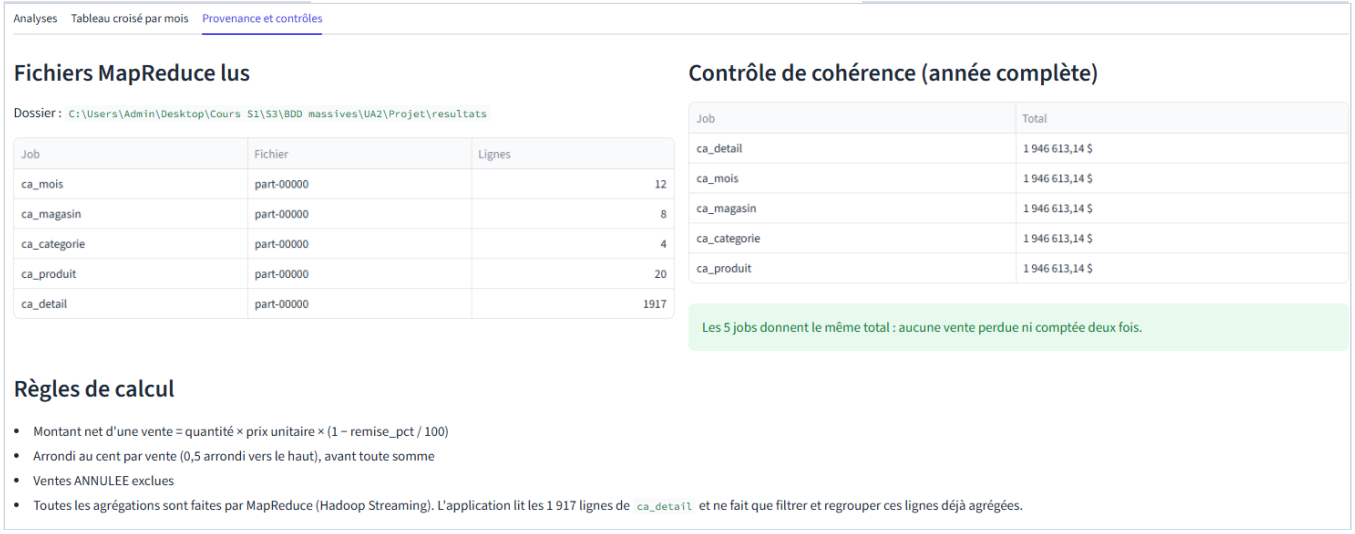
Ligne	janv.	févr.	mars	avr.	mai	juin	juil.	août	sept.	oct.	nov.	déc.	Total
Ottawa	**31 713**	**31 574**	**35 455**	**29 830**	**36 190**	**29 796**	**43 106**	**38 204**	**38 042**	**39 553**	**53 290**	**58 331**	**465 086**
M01	**21 821**	**18 027**	**20 164**	**19 312**	**19 764**	**16 140**	**23 906**	**23 220**	**22 894**	**22 304**	**31 542**	**37 601**	**276 695**
Electronique	10 727	9 665	9 446	7 753	7 029	5 967	9 214	10 079	9 725	7 137	12 133	15 234	114 110
Maison	6 359	4 404	5 374	6 845	7 203	5 335	9 518	6 916	7 826	7 639	11 439	12 920	91 778
Sport	2 759	2 199	3 422	2 523	3 485	2 821	3 220	3 779	3 423	4 850	4 970	6 151	43 604
Bureau	1 976	1 759	1 921	2 191	2 046	2 017	1 954	2 446	1 920	2 678	3 000	3 295	27 203
M02	**9 893**	**13 548**	**15 291**	**10 518**	**16 426**	**13 656**	**19 200**	**14 985**	**15 148**	**17 249**	**21 748**	**20 730**	**188 390**
Electronique	3 810	5 863	6 738	3 572	7 389	5 882	8 989	6 903	7 350	4 690	8 583	8 419	78 189
Maison	3 011	4 541	4 442	3 203	4 428	3 165	6 269	4 714	3 702	7 076	7 715	6 167	58 432
Sport	2 249	1 866	1 974	1 473	2 549	2 601	2 826	1 928	2 520	3 498	3 849	3 645	30 977
Bureau	824	1 279	2 137	2 271	2 060	2 007	1 117	1 440	1 575	1 984	1 601	2 498	20 793
Gatineau	**11 578**	**14 731**	**13 235**	**16 429**	**19 526**	**16 268**	**17 751**	**14 556**	**16 861**	**21 448**	**20 578**	**23 243**	**206 205**
M07	**11 578**	**14 731**	**13 235**	**16 429**	**19 526**	**16 268**	**17 751**	**14 556**	**16 861**	**21 448**	**20 578**	**23 243**	**206 205**
Electronique	3 944	5 840	7 994	7 150	8 190	5 161	5 317	6 279	7 193	10 359	9 003	8 090	84 520
Maison	4 334	4 506	2 558	5 472	6 905	6 491	8 071	4 245	5 455	5 680	6 219	8 271	68 209
Sport	1 768	2 682	1 468	2 640	2 965	3 166	2 564	2 344	2 768	3 139	3 397	4 175	33 076
Bureau	1 532	1 704	1 215	1 168	1 466	1 450	1 799	1 689	1 444	2 270	1 958	2 706	20 400
TOTAL GÉNÉRAL	**43 292**	**46 306**	**48 690**	**46 260**	**55 716**	**46 064**	**60 857**	**52 761**	**54 903**	**61 002**	**73 867**	**81 573**	**671 290**

Capture C10. *Onglet Tableau croisé par mois : villes en lignes, mois en colonnes, totaux*



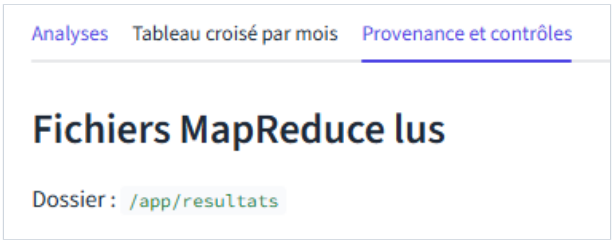
Ligne	janv.	févr.	mars	avr.	mai	juin	juil.	août	sept.	oct.	nov.	déc.	Total
M01	**21 821**	**18 027**	**20 164**	**19 312**	**19 764**	**16 140**	**23 906**	**23 220**	**22 894**	**22 304**	**31 542**	**37 601**	**276 695**
Electronique	10 727	9 665	9 446	7 753	7 029	5 967	9 214	10 079	9 725	7 137	12 133	15 234	114 110
Maison	6 359	4 404	5 374	6 845	7 203	5 335	9 518	6 916	7 826	7 639	11 439	12 920	91 778
Sport	2 759	2 199	3 422	2 523	3 485	2 821	3 220	3 779	3 423	4 850	4 970	6 151	43 604
Bureau	1 976	1 759	1 921	2 191	2 046	2 017	1 954	2 446	1 920	2 678	3 000	3 295	27 203
M02	**9 893**	**13 548**	**15 291**	**10 518**	**16 426**	**13 656**	**19 200**	**14 985**	**15 148**	**17 249**	**21 748**	**20 730**	**188 390**
Electronique	3 810	5 863	6 738	3 572	7 389	5 882	8 989	6 903	7 350	4 690	8 583	8 419	78 189
Maison	3 011	4 541	4 442	3 203	4 428	3 165	6 269	4 714	3 702	7 076	7 715	6 167	58 432
Sport	2 249	1 866	1 974	1 473	2 549	2 601	2 826	1 928	2 520	3 498	3 849	3 645	30 977
Bureau	824	1 279	2 137	2 271	2 060	2 007	1 117	1 440	1 575	1 984	1 601	2 498	20 793

Capture C11. *Accordéon ouvert : détail d'une ville par magasin et par catégorie*



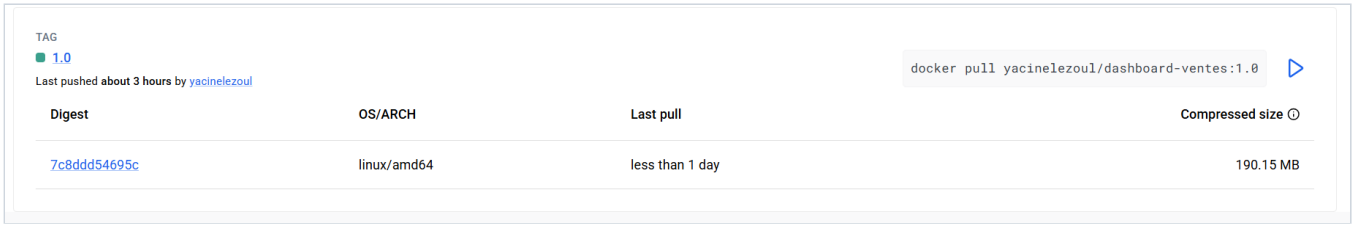
Capture C12. *Onglet Provenance et contrôles : fichiers `part-*` lus et totaux de contrôle égaux à ceux de ca_detail*

C.4 Image Docker



Capture C13. *Application lancée depuis l'image : les résultats sont lus dans `/app/resultats`*

Élément	Valeur relevée
Image publiée	`yacinelezoul/dashboard-ventes:1.0`
Digest après `docker push`	`sha256:412f047f4e70fa6342ab9c4e49842433291867713d7ebb8f4c8463065971caa7`
Après suppression locale et `docker pull`	Même digest ; « Downloaded newer image »
Taille	190,15 Mo compressée sur Docker Hub (capture C14), 899 Mo décompressée
Page publique	https://hub.docker.com/r/yacinelezoul/dashboard-ventes
Durée du build	172 s la première fois (102 s pour pip), 1,4 s ensuite grâce au cache



Capture C14. *Page Docker Hub du dépôt : étiquette 1.0, plateforme linux/amd64, 190,15 Mo compressée, commande `docker pull`*

Deux digests apparaissent, ce qui est normal : `docker push` affiche le digest de l'index de l'image (`412f047f…`), qui regroupe la version linux/amd64 et ses attestations de build ; Docker Hub affiche dans le tableau le digest de la seule version linux/amd64 (`7c8ddd54695c`). Le `docker pull` retrouve le digest de l'index publié.

C.5 Valeurs relevées dans les journaux

Journal	Valeur	Ce qu'elle prouve
Dépôt HDFS	md5 local = md5 HDFS = `4ae57ed472eb5c10dd07814deba2c37b` ; 24 001 lignes	Copie fidèle à l'octet près
fsck	1 bloc ; 2 blocs (1 048 576 + 937 219 octets) avec blocs de 1 Mo ; `Live_repl=2` ; HEALTHY	Découpage en blocs et réplication sur les 2 workers
Job 1	Map input 24 001, Map output 22 576, Reduce output 1 917 ; somme 194 661 314	Filtrage des ANNULEE et total exact en cents
Jobs 2 à 5	12, 8, 4 et 20 lignes ; somme de chaque sortie 194 661 314	Chaînage sans perte
Contrôle	md5 `db30e50fbe9042172383b551c5a4a634` pour ca_mois et le calcul direct	Le chaînage donne le même résultat qu'un calcul direct
Reducers 1, 2, 4	23, 27, 34 s ; Shuffled Maps 2, 4, 8 ; 1, 2, 4 fichiers part	Plus de Reducers n'accélère pas un petit job
Mappers	maps1 : 1 split ; défaut : 2 ; maps4 : 4 ; blocs de 1 Mo avec maps=1 : 2	`mapreduce.job.maps` est une indication, le bloc fixe un minimum
YARN	1 024 Mo par tâche ; job maps4 : 130 124 Mo·s et 90 vCore·s	Allocation minimale du conteneur
Cas de test	Map input 8, Map output 5, Reduce output 4 ; total 77 348 cents ; 3 × REUSSI	Résultat identique au calcul fait à la main